

Kafi — Nanny↔Family Flow Audit

Screen-by-screen audit of all app flows · 45 screens · 8 flow areas · generated 2026-07-20

Executive Summary

The nanny↔family product is **structurally sound and largely functional**. Every core loop was traced end-to-end in live mode and works: role/OTP auth, nanny onboarding + admin review, family discovery with a single canonical `MatchService`, server-authoritative free-view quotas and shortlists, realtime 1:1 chat, the trial accept/counter/decline/resign lifecycle, trial→hire conversion with badge retirement, and the support + (newly added) dispute chats with correct notify triggers. **No new Critical defects** were found on top of the already-merged live-mode remediation, and the one accepted launch blocker (deferred mock payments → contact-reveal denial) is unchanged and documented.

What remains is a long tail of **functional gaps, missing wiring, and polish** — 161 items. The single most consequential theme is that the **trial-offer chain is not wired end to end**: a family cannot actually start a trial from an applicant in-app, and several downstream pieces assume a path that never fires.

Findings roll-up

Severity	Count	Meaning
Critical	0	—
Major	25	Real functional gap / missing failure path a user will hit
Minor	99	Small correctness/UX gap, low blast radius
Polish	37	Cosmetic / consistency / nice-to-have
Total	161	across 45 screens / 8 flow areas

By flow area

#	Area	Major	Minor	Polish
1	Authentication & App Entry	3	9	6
2	Nanny Onboarding	0	14	6
3	Nanny Main App (home/jobs/applications/profile)	5	17	3
4	Family Onboarding, Profile & Settings	5	10	4
5	Family Discovery, Shortlist, Compare & Reveal	4	16	5
6	Family Applicants & Hiring (My Jobs)	5	9	3
7	Chat, Trial Lifecycle & Reviews	1	11	5
8	Support, Disputes, Notifications & Account	2	13	5

Top priorities to fix next

Ranked by user impact. Each links to the detailed section for `file:line` evidence.

1. **Major Family can't start a trial from an applicant.** `TrialController.offerTrial()` has **zero callers**, so no application ever reaches `trialOffered` and the nanny-side `trialOffered` branch is dead code. This breaks the intended apply→offer→trial handoff. (§6, §3)
2. **Major jobPostId is never threaded through offer→trial→hire.** Consequently the My-Jobs "Hired: {name}" pill can't fire in the normal flow, and the trial→application "hired" match falls back to the *first* application. (§6)
3. **Major Cover message is displayed to both roles but never collected** during apply — the field renders on the nanny apply UI and the family applicant card, but nothing captures it. (§3)
4. **Major Nanny sees stale application statuses all session.** My-Applications and Application-Detail have no realtime stream and no pull-to-refresh, and re-navigation doesn't reload the permanent controller. (§3)
5. **Major "Message Family" on application-detail appears dead** — it flips a hidden shell tab without popping the pushed route (the family path pops-to-root first; the nanny path doesn't). (§3)
6. **Major saveNanny writes the full model including server-owned stats**, unlike `saveFamily` which strips them — a post-approval profile edit can regress the dashboard counters. (§3)
7. **Major Compare and Video-Player screens are fully built but unreachable** — no navigation to `Routes.compare/Routes.videoPlayer` exists, so families can neither compare nannies nor watch intro videos (despite the card carrying `introVideoUrl`). (§5)
8. **Major Arabic is effectively unreachable.** The language switcher is gated behind `!embedInShell` (never true, since settings only renders embedded), and RTL is unwired (no `localizationsDelegates`) — so the full `ar_AE` translation set can't be used. (§4)
9. **Major Applicants & My-Jobs never render their error state** — a live read failure collapses to a misleading empty state, and applicant actions (`shortlist/decline/markAsViewed`) swallow write failures with no feedback. (§6)
10. **Major No in-app entry point to file fraud/abuse/no-show disputes** — the only filing path is the trial payment-issue dialog hardcoded to `DisputeCategory.payment`; the safety categories are defined and rendered but unreachable. (§8)
11. **Major "Support replied" / "Report resolved" notifications are no-ops when tapped** — those inbox entries use `type: systemAnnouncement` with no route, and `handleNotificationTap` has no branch to open the thread. (§8)
12. **Major Trial evaluation checklist is inert** — prominently rendered but the family can't tick items and `setOutcome` never passes a `TrialEvaluation`, so it always shows all-unchecked. (§7)

Cross-cutting themes

- **Trial-offer chain (highest value):** items 1–3, 12 — the offer→trial→hire path has missing wiring at nearly every seam.
- **Dead / unreachable surfaces:** Compare, Video-player, the dedicated `NannyEditProfileScreen`, in-app dispute filing, and the family-rating a nanny submits (surfaced only in the admin panel — a rating dead-end). (§5, §3, §8, §7)
- **Staleness / realtime:** nanny applications are one-shot `.get()` with no refresh; several list screens lack pull-to-refresh. (§3)
- **Data integrity:** `saveNanny` clobbers server `stats`; "Years exp" renders `experiences.length` (job count) not summed years across every profile surface. (§3, §5)
- **i18n / RTL:** Arabic unreachable (§4); hardcoded English strings in the trial screens, legal, and reason text despite `ar_AE` keys. (§7, §8)
- **Error paths:** splash offline cold-start hangs on an infinite spinner (no try/catch); applicant/my-jobs errors unrendered; assorted swallowed write failures. (§1, §6)
- **Visible polish:** OTP copy leaks `@phone/@time` literals; job-detail Save/Share are "(mock)" stubs; scattered placeholder names ("Sarah Reyes / Dubai"). (§1, §3)

Scope & Method

This audit walks **every user-facing screen** of the Kafi mobile app (`kafi_app/`, Flutter + GetX) and traces the end-to-end flows **between nannies and families** — onboarding, discovery, applying, chat, the trial lifecycle, hiring, reviews, payments, support and disputes. For each screen it records the current state and **every remaining task, adjustment, or polish item**, with `file:line` evidence.

It was produced by reading the actual view, controller, and service code (both the live `firebase/*` and `mock/*` service implementations behind `AppConfig.useMock`) screen by screen across the whole app.

Baseline. This audit is taken *after* the recent live-mode remediation — the 12-item Live-Mode audit backlog (all CRITICAL/MAJOR/MINOR items) is merged to `main`. Items already fixed there are **not** re-reported; this pass captures what remains on top of that baseline.

Severity legend

Badge	Meaning
Critical	Breaks a core flow or loses/handles data wrong in production. Fix before launch.
Major	A real functional gap or missing failure path a user will hit.
Minor	Small correctness/UX gap, low blast radius.
Polish	Cosmetic / consistency / nice-to-have.

Flow status per screen: **WORKING** (verified end-to-end) · **PARTIAL** (works but has gaps) · **MOCK-ONLY** (only the mock impl is complete) · **BROKEN** (a path fails).

Known, accepted limitation (not counted below)

Payments are **intentionally deferred** (product decision): the live build runs `useMockSubscription = true` and the mock entitlement is not synced to the server, so server-enforced **contact-reveal denies "paid" families**. This is the documented **#1 launch blocker** (`docs/PAYMENTS.md`) and is treated as a known state, not a new finding, throughout this report.

Detailed Findings by Flow Area

Each screen below carries its purpose, a flow-status verdict, and every remaining item with `file:line` evidence and a concrete fix.

1. Authentication & App Entry

Live mode is active: `AppConfig.useMock = false` (`app_config.dart:9`), so behavior below is judged against the Firebase impls. `AuthController` and all `I*Services` are permanent in `InitialBinding` (`initial_binding.dart:75-101`), which is why `splash/welcome/blocked` (routes with no binding) can `Get.find<AuthController>()` safely.

Splash Screen — `kafi_app/lib/views/shared/splash_screen.dart`

- **Purpose:** Cold-start gate; decides the first real screen (welcome / blocked / home / pending / next onboarding step) via `AuthController.bootstrapStartup`.

- **Flow status: PARTIAL** — decision logic is correct, but the startup path has no failure handling and can hang forever.
- **Findings:**
- **Major** No error handling on the entire startup path. `bootstrapStartup` (`auth_controller.dart:59-67`) and `_routeForUser` (`auth_controller.dart:262-284`) await network reads — `getCurrentUser` (Firestore `users/{uid}.get()`, `firebase_auth_service.dart:163`), `isUserBlocked`, `getNanny`, `getJobsByFamily` — with no `try/catch`. It is launched fire-and-forget from a post-frame callback (`splash_screen.dart:23-25`), so any throw (offline / Firestore unreachable at boot) becomes an unhandled async error and the user is stranded on the infinite spinner with no retry. Evidence: `splash_screen.dart:29-59` renders only a `CircularProgressIndicator` — there is no error/timeout/retry state. Fix: wrap `bootstrapStartup` in `try/catch`, and on failure show a retry affordance (or route to welcome with a snackbar) instead of leaving the spinner.
- **Minor** `_routeForUser` only reaches `browse/familyForm/nannyStart` after several sequential awaits; on a slow cold start the splash spinner can persist for seconds with no "still working" feedback. Evidence: `auth_controller.dart:262-283`. Fix: acceptable, but consider a minimum-timeout fallback so a stalled Firestore call surfaces the error path above.

Welcome Screen — `kafi_app/lib/views/auth/welcome_screen.dart`

- **Purpose:** Role picker (Nanny vs Family) and app entry point after splash decides "no session".
- **Flow status: WORKING** — both cards and the bottom links route correctly and prime the chosen role.
- **Findings:**
- **Polish** Duplicate navigation surface: the two `_RoleCards` (`welcome_screen.dart:59-83`) and the bottom "Have an account?" links (`welcome_screen.dart:94-114`) call the identical `prepareXLogin()` + `Get.toNamed` for the same targets. The unified phone-OTP flow means "sign up" and "sign in" are the same action, so the bottom row is redundant. Fix: keep for discoverability or collapse — no functional issue.
- **Polish** `prepareNannyLogin/prepareFamilyLogin` runs here on tap (`welcome_screen.dart:13,18`) and again in each login screen's `initState` (`login_nanny_screen.dart:30`, `login_family_screen.dart:25`), so role-set + `requestStartupPermissions` fire twice per entry. It is idempotent and permission requests are guarded, so harmless. Fix: drop the welcome-side calls or the `initState` calls to remove the double-fire.

Login — Nanny — `kafi_app/lib/views/auth/login_nanny_screen.dart`

- **Purpose:** Phone entry (nanny country codes) → Send OTP → OTP screen.
- **Flow status: WORKING** — validation, loading, send, and navigation are all wired through `AuthController.sendOtpAndNavigate`.
- **Findings:**
- **Minor** Reimplements the country+phone field inline (`_phoneInput`, `login_nanny_screen.dart:149-204`) instead of reusing the shared `KafiPhoneInput` widget that the family screen uses. Duplicated logic + a visibly different input style between the two login screens. Fix: use `KafiPhoneInput` (it already accepts `countryOptions`) for a single source of truth.
- **Minor** Hardcodes its 13-country list inline (`login_nanny_screen.dart:21-24`) while `AuthConstants.nannyCountryCodes` (`auth_constants.dart:16-30`) already defines the same set (and is used by `nanny_info_screen.dart:27`). Two sources of truth that can drift. Fix: derive the dropdown from `AuthConstants.nannyCountryCodes`.
- **Minor** No back affordance to return to Welcome to switch role. The OTP screen has a back row but neither login screen does; on iOS (no system back) a user who tapped the wrong role is stuck. Evidence: `login_nanny_screen.dart:34-98` (no `AppBar/back`). Fix: add a back button consistent with `otp_verify_screen.dart:22-36`.

- **Polish** The displayed `formattedPhone` uses the raw trimmed text (`auth_controller.dart:38-39`) including a leading trunk 0, but the number actually sent to Firebase strips that 0 (`validators.dart:19-22`, `firebase_auth_service.dart:61-65`). A user typing 050... sees +971 050... on the OTP screen while the SMS went to +971 50... Fix: format the display from the sanitized national number.

Login — Family — `kafi_app/lib/views/auth/login_family_screen.dart`

- **Purpose:** Phone entry (family/GCC/international codes) → Send OTP → OTP screen.
- **Flow status:** **WORKING** — uses shared `KafiPhoneInput`; same controller path as nanny.
- **Findings:**
 - **Minor** Hardcodes its 25-country list inline (`login_family_screen.dart:76-82`) while `AuthConstants.familyCountryCodes` (`auth_constants.dart:33-65`) defines the same set and is otherwise **entirely unused** across the app (grep: no references). Dead constant + drift risk. Fix: source the dropdown from `AuthConstants.familyCountryCodes`.
 - **Minor** No back affordance to Welcome (same as nanny screen). Evidence: `login_family_screen.dart:29-122`. Fix: add a back button.
 - **Polish** The shared `phoneController` (`auth_controller.dart:33`) is not cleared when `prepareFamilyLogin/prepareNannyLogin` runs, so a number typed under one role persists if the user backs out and switches role. Only `countryCode` is reset (`auth_controller.dart:72, 80`). Fix: clear `phoneController` (and `otpCode/errors`) in the prepare methods.

OTP Verify Screen — `kafi_app/lib/views/auth/otp_verify_screen.dart`

- **Purpose:** 6-box OTP entry, live countdown, verify → finalize → role routing; resend / change-number.
- **Flow status:** **PARTIAL** — verify/routing works, but a placeholder-substitution bug corrupts the on-screen copy and resend UX is degraded.
- **Findings:**
 - **Major** The `@phone` placeholder leaks into the subtitle. The code splits the resolved translation on `controller.formattedPhone` instead of the literal token, but the string still contains `@phone` (it uses `.tr`, not `.trParams`). "Enter the 6-digit code we sent to\n@phone".split("+971 50...") finds no match, so `.first` returns the whole string and the bold phone is appended after it → the screen renders ...sent to / @phone+971 50.... Evidence: `otp_verify_screen.dart:52`; string at `en_us.dart:71`. Fix: `AppStrings.otpEnterSub.tr.split('@phone').first` (split on the literal token), or switch to `trParams`.
 - **Major** Identical bug on the expiry line: `AppStrings.otpExpires.tr.split(label).first` where `label` is the resolved time (e.g. 5:00). The string is "Code expires in @time", so it renders Code expires in @time5:00. Evidence: `otp_verify_screen.dart:82`; string at `en_us.dart:72`. Fix: split on '@time' (or use `trParams`). (Note: the "OTP sent" snackbar does this correctly via `trParams` at `auth_controller.dart:114`, confirming these two are the outliers.)
 - **Minor** Resend is disabled for the full 5-minute OTP lifetime. `canResendOtp => otpSecondsLeft.value <= 0` (`auth_controller.dart:125`), and `otpSecondsLeft` counts down the 300s expiry — there is no separate cooldown. `AuthConstants.otpResendCooldownSeconds = 60` (`auth_constants.dart:6`) is defined but **never referenced** (grep-confirmed), so the intended 60s resend window is dead. A user who never receives the SMS must wait 5 min (or use "Change number") before the Resend link activates. Evidence: `otp_verify_screen.dart:115, 120`. Fix: gate Resend on a 60s cooldown timer using `otpResendCooldownSeconds`, independent of the expiry countdown.
 - **Minor** OTP boxes desync from the controller on resend. The 6 boxes are local `TextEditingController`s in `_BigOtpBoxesState` (`otp_verify_screen.dart:166-220`), only pushed up via `onChanged`. `resendOtp` clears `controller.otpCode` to '' (`auth_controller.dart:135`) but cannot clear the boxes, so the old digits stay visible while the model is empty; a subsequent Verify

sees length 0 and shows "incorrect" over full-looking boxes. Fix: make `_BigOtpBoxes` observe `controller.otpCode` (clear boxes when it resets), or expose a clear hook the controller can call.

- **Polish** OTP entry lacks common conveniences: no backward focus-jump on delete, no paste-to-distribute of a 6-digit code, and no auto-submit when the 6th digit is entered. Only forward advance is handled. Evidence: `otp_verify_screen.dart:211-214`. Fix: add backspace-to-previous and (optionally) auto-verify on completion.
- **Polish** `otpError` is not cleared while the user edits the code — a prior "incorrect/expired" error persists under the boxes until the next Verify. Evidence: error shown at `otp_verify_screen.dart:62-74`; `otpError` only reset inside `verifyOtpAndNavigate` (`auth_controller.dart:152,168`). Fix: clear `otpError` in the boxes' `onChanged`.

Blocked Screen — `kafi_app/lib/views/shared/blocked_screen.dart`

- **Purpose:** Terminal screen for an admin-blocked account; logout-only, no way back into the app.
- **Flow status:** **WORKING** — `PopScope(canPop:false)` blocks system back, the only action is logout → `signOut` → `offAllNamed(welcome)`, and it is reachable both from startup routing and the live block watcher.
- **Findings:**
- **Minor** Block-watch lifecycle gap (controller-side, surfaces here): `signOut` cancels `_blockSub` (`auth_controller.dart:333`) but `deleteAccount` does not (`auth_controller.dart:337-346`), so after account deletion the watcher keeps listening on a now-deleted role doc and could fire `_routeForUser` on a dead user. Fix: cancel `_blockSub` in `deleteAccount` too.

Cross-cutting / flow checks

- Role routing (splash + post-verify): **WORKING** — nanny → approved:`nannyHome`, pending/rejected:`nannyPending`, draft: first missing step (`_nannyStartRoute`, `auth_controller.dart:289-304`); family → browse if ≥1 job else `familyForm` (`auth_controller.dart:271-282`); blocked → `blocked`. Verified end to end.
- Live OTP send/verify: **WORKING** — `verifyPhoneNumber` (60s timeout) → `signInWithCredential` (`firebase_auth_service.dart:67-105`); `finalizePhoneRegistration` does an idempotent merge + skeleton create that never resets a returning user (`firebase_auth_service.dart:108-152`).
- Interrupted-signup durability: **WORKING** — `verificationId` and pending role are mirrored to `SharedPreferences` so an app-kill between "code sent" and "verify"/"finalize" resurrects the right role instead of the `nanny` default (`firebase_auth_service.dart:26-56, 94-105`).
- Error mapping / states: **WORKING** — comprehensive `FirebaseAuthException` → localized-string mapping incl. region-disabled and network cases (`auth_controller.dart:194-246`); all referenced keys exist in `en_us.dart` (`authOtpRateLimited`, `authQuotaExceeded`, `authSmsRegionDisabled`, `authNoAccount`, `authReauthRequired`, `authOtpSendFailed`). Loading spinners are bound on both Send-OTP and Verify buttons.
- Lockout / rate-limit: GAP (minor) — enforced **server-side only** by Firebase (`too-many-requests/quota-exceeded` mapped); there is no client-side attempt cap, so wrong-OTP retries are unbounded until Firebase throttles. Acceptable but worth noting.
- Live mid-session block/unblock: **WORKING** — `watchBlocked` stream drives `_startBlockWatch` (`auth_controller.dart:308-320`); GAP (minor): the `.listen` has no `onError`, so a Firestore stream/permission error on the watcher is unhandled.
- `AppConfig.useMock` behavior delta: only the "OTP sent" snackbar changes (mock shows the literal code `MockConstants.otp`, live shows "sent to {phone}", `auth_controller.dart:110-115`) plus the service impl swap in `InitialBinding`. Flow shape is identical. `useMock` is currently `false` (live).
- `AuthBinding` redundancy: GAP (minor) — `AuthBinding` (`auth_binding.dart`) only puts `AuthController` "if not registered", but it is already `permanent` in `InitialBinding`.

(`initial_binding.dart:99`), so the binding attached to `loginNanny/loginFamily/otpVerify/deleteAccount` routes (`routes.dart:99-101,142`) is always a no-op. Harmless dead wiring.

- **Localization:** GAP (minor, not live) — `main.dart:52-53` pins `locale/fallbackLocale` to `en_US` even though `ar_AE` is in `supportedLocales`, so the app ships English-only with no in-app language switch; and `ar_ae.dart` is missing `otpEnterSub/otpExpires/authOtpSentBody`, which would render raw keys if Arabic were ever activated. The `@phone/@time` leak above is locale-independent.

2. Nanny Onboarding

Audited against the current `main` (live mode: `AppConfig.useMock = false` — `app_config.dart:9`). Mock-only branches in the controller are inert; live paths upload to Firebase Storage and write Firestore. The recently-merged CRITICAL/MAJOR fixes (gallery video-duration probe, `documents/` storage path, staged-then-upload docs, C5 URL split to the private subcollection, reference flags) are present and verified. No Critical or true Major regressions remain; the flow works end to end. Findings below are the remaining minor / polish items.

Nanny Info (Step 1 — personal/demographics) — `kafi_app/lib/views/nanny/nanny_info_screen.dart`

- **Purpose:** Collects identity, visa, work location/prefs, personal, health, comfort, religion, emergency contact and bio; save-on-Next.
- **Flow status:** **WORKING** — full required-field validation, DOB 18+ gate, salary range, and save/hydrate all function.
- **Findings:**
 - **Minor** Emergency-relationship dropdown shows "Spouse / Partner" by default but never writes it to the controller, so `emergencyRelCtrl.text` stays empty and validation fires `nannyEmergencyRelRequired` even though a value appears selected. Evidence: dropdown `nanny_info_screen.dart:630-640` + `_relValue()` `nanny_info_screen.dart:656-660` (display-only fallback, no write); guard `nanny_profile_controller.dart:442`. Fix: seed `emergencyRelCtrl.text` to the first option on init (or write the fallback in `onChanged/_relValue`) so the displayed value matches the stored value.
 - **Minor** Nationality is pre-seeded to 'Filipino' while the other required selections were deliberately reset to "unset"; a non-Filipino nanny who overlooks the field silently saves `Filipino` and the `isEmpty` check never catches it. Evidence: `nanny_profile_controller.dart:61`; validation `nanny_profile_controller.dart:423`. Fix: start `nationality` empty (like languages/visa/emirates) so the picker forces an explicit choice, or accept the default intentionally and drop the dead `isEmpty` check.
 - **Minor** Current-area location picker does not reflect a hydrated draft value on a fresh app-restart resume: it captures `initialValue` once in `initState` and is not wrapped in `Obx`, while hydration is async (after first build). The underlying `currentAreaCtrl.text` is retained (so validation passes and no data is lost), but the field looks empty. Evidence: `nanny_info_screen.dart:375-381`; picker `kafi_location_picker.dart:88-91` (`initState`) + `:94-102` (`didUpdateWidget` only reacts to a changed `initialValue`); async hydrate `nanny_profile_controller.dart:310`. Fix: key/rebuild the picker off the hydrated value (e.g. `Obx/ValueKey(currentArea)`), or push hydration before first paint. (Edit-mode entry is unaffected — the permanent controller is already hydrated.)
 - **Polish** `currentStep` Rx is written (2→5) on each save but never read anywhere; the header renders a hardcoded `step:`. Dead state. Evidence: `nanny_profile_controller.dart:48,505,667,726,792`. Fix: drive `KafiFormHeader.step` from `currentStep` or remove the field.

Nanny Media (Step 2 — photos + intro video) — `kafi_app/lib/views/nanny/nanny_media_screen.dart`

- **Purpose:** Upload 3–5 photos and a ≤60s intro video; photos/video upload to Storage on pick, URLs persisted on Next.
- **Flow status:** **WORKING** — min-3/max-5 photo gating, camera+gallery video duration enforcement (camera `maxDuration` + gallery `_videoExceedsLimit` probe), real `video_player` preview with graceful failure fallback.
- **Findings:**
 - **Minor** No client-side byte-size cap on photos or video (only photo compression and video *duration*). The whole video is loaded into memory via `readAsBytes()` before upload; a 60s HD/4K clip can be 100 MB+, risking OOM / very slow uploads on low-end devices. Evidence: video `nanny_profile_controller.dart:607`; only duration checked `:625–636`; photo pick has no size guard `:532–552` (contrast docs' 25 MB cap `:876–879`). Fix: add a byte-size ceiling for video (and optionally stream/resumable upload) mirroring the doc guard.
 - **Minor** Removing a photo (or abandoning after picking) leaves orphaned files in Storage — `removePhoto` only mutates the list, and photos upload immediately on pick before the profile is saved on Next. Video overwrites a fixed path (no video orphan) but removing the video doesn't delete it either. Evidence: `nanny_profile_controller.dart:560–562` (`removePhoto`), `:542–552` (upload on pick), `:357–361` (video clear, no delete). Fix: call `_storageService.deleteFile` on remove, or defer photo uploads until Next.
 - **Polish** `introVideoName` is not hydrated on resume (`_hydrate` sets `introVideoUrl` only), so a resumed video preview shows the generic fallback name. Evidence: `nanny_profile_controller.dart:343`. Fix: persist/hydrate the name (or derive it from the URL).
 - **Polish** A long video upload surfaces only the footer button spinner — no blocking loader like the docs step, so feedback is weak during a multi-second upload. Raw `e.toString()` is shown on failure instead of a localized message. Evidence: `nanny_profile_controller.dart:600` (isLoading only), `:554, 616` (`e.toString()`). Fix: reuse `_showBlockingLoader` for media and swap raw errors for a localized string.

Nanny Experience (Step 3 — work history) — `kafi_app/lib/views/nanny/`

`nanny_exp_screen.dart`

- **Purpose:** Add/edit inline work-experience cards (title, employer, location picker, from/to dates, children, duties, reason); optional step.
- **Flow status:** **WORKING** — date-range picker constraints + belt-and-suspenders `experienceDates` validation; add/remove instant-save.
- **Findings:**
 - **Minor** Field edits inside a card are not instant-saved — only add/remove persist. `onChanged` mutates the in-memory `RxList` element; edits are written only on Next, so abandoning after editing (without Next) keeps the card but loses the typed fields. Evidence: `nanny_exp_screen.dart:43`; `_persistExperiences` fires only from add/remove `nanny_profile_controller.dart:676–700`. Fix: debounce-persist on card change, or document that edits save on Next (consistent with the rest of the step).
 - **Minor** On app-restart resume the experience step is silently skipped — `_nannyStartRoute` jumps a draft straight from media to docs and never lands on exp/refs. Data already entered is retained, but the nanny is never re-prompted to finish it. Evidence: `auth_controller.dart:298–303`. Fix: acceptable if experience is intentionally optional; otherwise include exp/refs completeness in the resume routing.
 - **Polish** No content validation on experience cards — a card added with empty employer/city/children/duties saves fine (only date order is checked). Evidence: `nanny_profile_controller.dart:710–716`. Fix: require employer + at least duties when a card exists, or drop empty cards on save.

Nanny References (Step 4 — references) — `kafi_app/lib/views/nanny/`

`nanny_refs_screen.dart`

- **Purpose:** Declare callable references (`hasReferences`), add reference cards, and a `commitsToShare` commitment checkbox.
- **Flow status:** **WORKING** — the recently-added "Yes but empty list" guard is correct, and the two flags persist together on both instant-save and Next.
- **Findings:**
 - **Minor** `commitsToShare` is saved but never required, despite the UI stressing "You must follow through — families trust what you declare here." A nanny who declares references can proceed with the commitment box unchecked. Evidence: checkbox `nanny_refs_screen.dart:241-246`; `saveRefsAndNext` guards only `hasReferences && references.isEmpty` `nanny_profile_controller.dart:775-778`. Fix: require `commitsToShare` when `hasReferences` is true (or make it clearly optional in the copy).
 - **Minor** Selecting "No references" neither hides nor clears already-added reference cards; the card list renders regardless of the toggle, and Next persists `hasReferences=false` alongside a non-empty `references` list — an inconsistent stored state. Evidence: cards render unconditionally `nanny_refs_screen.dart:61-73`; save writes both `nanny_profile_controller.dart:781-785`. Fix: clear `references` (or hide the section) when the nanny switches to "No".
 - **Polish** No per-field validation on reference cards — empty `canConfirm/city` and `yearsWorked = 0` are accepted. Evidence: `_emit` builds the model verbatim `nanny_refs_screen.dart:301-310`; the "Yes" guard only checks list non-emptiness. Fix: require at least `canConfirm` (and a real `yearsWorked`) per card.
 - **Minor** Same instant-save gap as experience: card field edits persist only on Next; toggling `hasReferences/commitsToShare` alone isn't persisted until an add/remove or Next. Evidence: `nanny_refs_screen.dart:45-54, 68` + `_persistReferences` `nanny_profile_controller.dart:750-764`. Fix: persist on toggle change, or accept save-on-Next.

Nanny Documents (Step 5 — KYC) — `kafi_app/lib/views/nanny/nanny_docs_screen.dart`

- **Purpose:** Stage passport/visa (required), Emirates ID (conditional), training cert + police clearance (optional); upload-on-submit; the "Submit for review" gate lives here.
- **Flow status:** **WORKING** — staged-then-uploaded on submit, 25 MB cap aligned with storage rules, correct `documents/` path, private-subcollection URL write (C5), and required-docs guard.
- **Findings:**
 - **Minor** "Too large" message says "under 10 MB" but the actual cap is 25 MB, so a 12–24 MB file is accepted yet the error copy (if ever shown) misstates the limit. Evidence: cap `nanny_profile_controller.dart:805, 876-879`; string `l10n/locales/en_us.dart:835`. Fix: update `docTooLarge` copy to 25 MB (and mirror in `ar_ae.dart:417`).
 - **Minor** `docPickFailed` copy says "pick a PDF or image (JPG/PNG)" but the picker accepts `FileType.any` (videos, Word docs, etc. — intentional per admin verification). Message contradicts behavior. Evidence: picker `nanny_profile_controller.dart:851-858`; string `l10n/locales/en_us.dart:834`. Fix: generalize the message ("choose a PDF, image or video").
 - **[Minor/Polish]** No preview/thumbnail of a picked or previously-uploaded document, and no explicit remove — re-tapping simply replaces. Hydrated docs also carry no `url` (stripped for C5), so there is nothing to preview on resume even if a viewer existed. Evidence: `_doc` → `KafiDocItem` shows status text only `nanny_docs_screen.dart:130-144`, `kafi_doc_item.dart:30-113`. Fix: show a filename/thumbnail of the staged file and an explicit remove control.
 - **Minor** On resubmit after a partial rejection, a previously-rejected document that isn't re-picked keeps `status=rejected` and its per-doc `rejectionReason` — `submitForReview` only maps `uploaded→reviewing`, and only the top-level nanny `rejectionReason` is cleared server-side. The pending screen can then show a "Rejected" doc under an overall pending profile. Evidence: `nanny_profile_controller.dart:1031-1037`; server clears only top-level fields

`firestore_user_service.dart:97-107`. Fix: reset stale `rejected` docs to `reviewing` (and clear their reason) on resubmit, or require re-upload of any rejected doc.

Nanny Pending / Rejected (Step 6 — awaiting review) — `kafi_app/lib/views/nanny/nanny_pending_screen.dart`

- **Purpose:** Post-submit status screen — pulsing pending hero, rejected hero + admin reason, per-doc/intro-video status list, and update-docs / resubmit actions.
- **Flow status:** **WORKING** — live status watch flips to home on approval and re-renders + snackbars on live rejection; resubmit re-runs `submitForReview`, and the server clears the stale top-level reason.
- **Findings:**
 - **Minor** "Resubmit" does not force re-upload of a rejected document: `hasRequiredDocs` only checks `status != missing`, and a `rejected` doc counts as present, so a nanny can resubmit the same unchanged (still-rejected) document and re-enter the queue. Evidence: `nanny_pending_screen.dart:220-233` (resubmit → `submitForReview`); guard `nanny_profile_controller.dart:123-128`. Fix: treat `rejected` as "needs re-upload" for the required-docs gate.
 - **Polish** The "Update documents" link is shown for a still-`pending` (not-yet-reviewed) nanny too, allowing a re-submit that resets `submittedAt` and re-queues her before review. Evidence: `nanny_pending_screen.dart:50-54, 200-218`. Fix: hide update/resubmit while `pending` (show it only for `rejected`), or make it idempotent.

Cross-cutting / flow checks

- Mandatory-field validation per step: **WORKING** — Step 1 has a full ordered `validatePersonalInfo` (`nanny_profile_controller.dart:417-448`), Step 2 `validateMedia` (:641-648), Step 5 required-docs gate (:1019-1023). Steps 3–4 are optional by design (only date-order / "Yes-but-empty" guards).
- DOB age ≥ 18: **WORKING** — picker `lastDate = now - 18y` (`nanny_info_screen.dart:66`) and `Validators.dateOfBirth` rejects <18 / future dates (`validators.dart:42-48`).
- Emergency contact: **WORKING** (name/relationship/phone all validated `nanny_profile_controller.dart:441-445`) — except the dropdown-default display gotcha above.
- Location-picker persistence: **WORKING** — `currentLocationPicked` saved as `currentLocation` and hydrated back (`nanny_profile_controller.dart:473, 311`); reference/experience `GeoLocation` round-trips via `nanny_map_codec.dart`. GAP: display refresh on async resume (see Step 1).
- Photo/video limits + failure handling: **WORKING** — 3–5 photos, ≤60s video (camera + gallery), try/catch snackbars. GAP: no byte-size cap / full in-memory read for video; orphaned Storage on remove (see Step 2).
- Instant-save vs save-on-Next: **WORKING** — Steps 3/4 instant-save on add/remove; all else save-on-Next. Consistent, though card *field* edits are save-on-Next only.
- Submit-for-review → status flip → admin-notify: **WORKING** app-side — `submitForReview` uploads staged docs, flips `status=pending`, and `submitNannyForReview` writes `submittedAt` + deletes the prior verdict (`firestore_user_service.dart:97-107`). Admin-notify is a server Cloud Function (not in this repo) — not verifiable here.
- Rejected-nanny resubmit path: **WORKING** — pending screen renders reason + resubmit; server clears the top-level reason on resubmit. GAPs: rejected per-doc status/reason not reset; same doc can be resubmitted unchanged (see Steps 5–6).
- Resume after app restart (draft hydration): **WORKING** for controller-bound fields via `_hydrate` (`nanny_profile_controller.dart:292-351`) and `_nannyStartRoute` (`auth_controller.dart:289-304`). GAPs: resume routing skips exp/refs; location-picker display; `introVideoName` not hydrated.
- `AppConfig.useMock` behavior: VERIFIED — `useMock=false`, so the mock-only branches (local-path

photo/video :542-545, 602-606, `_bootstrap` auto-submit :203-205, mock auto-approve timer) are inert; live Storage upload + Firestore writes are the active paths.

3. Nanny Main App (Home, Jobs, Applications, Profile)

Audited against the CURRENT state on `main` (`AppConfig.useMock = false` → LIVE Firestore; `useMockSubscription = true` → subscription only is mocked). Prior live-mode CRITICAL/MAJOR fixes (error/retry states, notif-dot gating, withdraw confirms, apply dup-guard, storage paths) are already in and verified — not re-reported.

Nanny shell — `kafi_app/lib/views/nanny/nanny_shell_screen.dart` (+ `controllers/nanny_shell_controller.dart`)

- **Purpose:** Bottom-nav host: tab 0 Dashboard, 1 Jobs, 2 Messages (family `ChatScreen` embedded), 3 Profile (family `SettingsScreen` embedded, role-branched).
- **Flow status: WORKING** — `NannyShellController.tabIndex` drives an `Obx`, `goToTab` clamps 0-3, `KafiBottomNav.activeIndex` highlights correctly; entering via `/nanny-jobs` starts on tab 1 (`onInit`).
- **Findings:**
 - **Polish** Body is `_tabBody(tab)` (one live tab, not `IndexedStack`), so every tab switch tears down and rebuilds the outgoing tab — scroll position is lost and each tab re-runs its `initState` side effects. Evidence: `nanny_shell_screen.dart:40-41`. (This rebuild is also what makes the `ChatScreen` deep-link consume on re-entry — see application-detail.) Fix: accept the tradeoff or switch to an offstage-preserving host; at minimum keep list scroll offsets.

Nanny dashboard — `kafi_app/lib/views/nanny/nanny_dashboard_screen.dart`

- **Purpose:** Home — profile hero (verified badge + server stats), employment status card (hired/on-trial + resign), profile-quality score/checklist, "Jobs for you" preview.
- **Flow status: WORKING** — stats are genuinely server-maintained (`functions/src/triggers/stats.ts` owns `stats.shortlists/profileViews/hiresCount/averageRating`) and read from `nanny.stats`; resign and status-card→Messages both work (tab 0 has no pushed route over it).
- **Findings:**
 - **Minor** Null-profile placeholder shows a hardcoded demo identity "Sarah Reyes / Live-in Nanny · Dubai" before `nanny.value` loads. Evidence: `nanny_dashboard_screen.dart:79-82`. Fix: use the auth user's real name or a neutral skeleton — never a fake name in a live build.
 - **Minor** "🔍 Jobs for you" preview is `filteredJobs.take(2)` and is NOT ranked by match, unlike the Jobs tab which sorts by score desc — so the "for you" preview may not show the best two matches. Evidence: `nanny_dashboard_screen.dart:512 + :528-533`. Fix: sort by `calculateJobMatch` desc before `take(2)`.
 - **Minor** Quality checklist doesn't match the actual score formula: it lists "Kafi Verified badge" and "Multiple photos added" (:424-428) which `calculateProfileScore` does NOT count, while the formula's experiences/references/passport inputs aren't shown as rows. Only the police/cert +10/+7 bonuses line up. Evidence: `nanny_dashboard_screen.dart:421-437` vs `nanny_profile_controller.dart:1056-1073`. Fix: align checklist rows with the scoring inputs (or relabel as generic tips).
 - **Minor** Comment claims `profileScore` is a "Real server-maintained score" but it is client-computed by `calculateProfileScore()` and client-written via `saveNanny` (no server function exists for it, unlike `stats`). Evidence: `nanny_dashboard_screen.dart:372-374`. Fix: correct the comment, or make the score server-owned for consistency.

Jobs home — `kafi_app/lib/views/nanny/jobs_home_screen.dart` (+ `controllers/job_post_controller.dart`, `services/match_service.dart`)

- **Purpose:** Browse active jobs with real per-nanny match scoring, search, filter chips, and applied/viewed/offers stats.
- **Flow status:** **WORKING** — match scoring + best-first sort, reactive client search, and error/empty/retry states are all correct; single `MatchService` is the shared scorer.
- **Findings:**
 - **Minor** Filter chips trigger a full network reload (`applyFilter` → `loadJobs`) even though `browseJobs` only queries `status==active` (+optional emirate) and IGNORES `jobType/duties` server-side — so each chip tap re-fetches the identical ~50 docs, filters client-side, and flashes the full-screen spinner (`isLoading` replaces the list). Evidence: `jobs_home_screen.dart:213-221` → `job_post_controller.dart:45-48` → `firestore_job_service.dart:85-94`. Fix: filter client-side only; the data is already in `allJobs`.
 - **Minor** Header "N jobs matching" uses `allJobs.length` (unfiltered) while the list shows client-filtered/searched results — count mismatches whenever a chip or search is active. Evidence: `jobs_home_screen.dart:70-73`. Fix: show `filteredJobs.length`.
 - **Minor** `browseJobs` uses `.limit(50)` with NO `orderBy` — ordering is undefined and only the first 50 active jobs are ever reachable (no pagination). Evidence: `firestore_job_service.dart:86-93`. Fix: add `orderBy(createdAt desc)` + pagination.
 - **Minor** No pull-to-refresh; new jobs only appear via a filter-chip reload or relaunch. Evidence: `jobs_home_screen.dart:250-283` (no `RefreshIndicator`). Fix: wrap the list in `RefreshIndicator` → `loadJobs`.

Job detail + apply — `kafi_app/lib/views/nanny/job_detail_screen.dart` (apply flow → `views/family/smart_match_screen.dart` → `controllers/application_controller.dart`)

- **Purpose:** Full job detail (family card, match ring + 11-factor breakdown, details/requirements/benefits/visa) with an Apply CTA that routes through the smart-match confirm screen.
- **Flow status:** **PARTIAL** — detail renders correctly and apply's guards work, but the cover-message feature is never wired.
- **Findings:**
 - **Major** Cover message is never collected. Apply chain `job_detail_screen.dart:353` → `smart_match_screen.dart:356` `controller.applyToJob(job.id)` passes NO `coverMessage`, and there is no cover-message input anywhere in the nanny UI — yet the field is stored (`firestore_application_service.dart:84`) and DISPLAYED to both sides (`my_applications_screen.dart:180-185`, `application_detail_screen.dart:579-585`, `family_applicants_screen.dart:180-182`). Result: every live application has an empty cover message and the "Cover Message" card never renders. Fix: add an optional cover-message `TextField` on the smart-match apply step and pass it to `applyToJob`.
 - **Minor** Share & bookmark hero actions are non-functional mocks that surface literal "(mock)" text to users in a live build, and are hardcoded English. Evidence: `job_detail_screen.dart:88-90`. Fix: implement or hide them; never show "(mock)" in prod.
 - **Minor** Hero title 'Job Details' is a hardcoded literal, not localized. Evidence: `job_detail_screen.dart:87` (same for the share/save snackbar strings).
 - **Minor** When `nanny == null` (controller absent / profile not yet loaded) the match hard-codes 0 and empty factors → a red "0% / Low match" with no rows. Evidence: `job_detail_screen.dart:20-28`. Fix: show a loading/neutral state instead of a fake 0%.
 - **Polish** `_requirementsSection/_benefitsSection` still render a titled card when `job.duties/job.benefits` are empty (header, no rows). Evidence: `job_detail_screen.dart:238-253`. Fix: hide the card when its list is empty.

- Dup-guard (client fast-path + deterministic doc id + `already_applied`): **WORKING** (`application_controller.dart:106-112`, `firestore_application_service.dart:47-58`). Active-trial guard: **WORKING** (`application_controller.dart:88-98`). **2-job cap**: enforced at TRIAL ACCEPT (`TrialController._nannyAtJobCap`: active hires + accepted/active trials $\geq 2 \rightarrow$ `nannyJobCapReached`), NOT at apply — correct by design (applying is unlimited; the cap binds at commitment). Evidence: `trial_controller.dart:302-306` + `:518-530`. **WORKING**.

My applications — `kafi_app/lib/views/nanny/my_applications_screen.dart` (+ `controllers/application_controller.dart`)

- **Purpose**: Nanny's sent-application tracking list with status badges, search, and in-card withdraw.
- **Flow status**: **WORKING** content, but a shared staleness gap undermines it.
- **Findings**:
 - **Major** No realtime, no pull-to-refresh, and no reload on re-entry. `ApplicationController` is `permanent`, so `loadApplications` runs once at init / on auth change / after the nanny applies — re-navigating to the route does NOT re-run it, and there's no `RefreshIndicator` or `watch*` stream (interface is one-shot `getApplicationsForNanny`). Family-side transitions (`viewed`→`shortlisted`→`trialOffered`→`declined`→`hired`) never surface during a session; even the "application viewed/declined" notification deep-link just `Get.toNamed(Routes.nannyApplications)` onto the already-initiated controller (`notification_controller.dart:166-172`), showing stale data. Evidence: `application_controller.dart:46-56` + `:114`; `my_applications_screen.dart:29-47` (no refresh). Fix: add a live stream (`watchApplicationsForNanny`) or a `RefreshIndicator` + reload-on-entry.
 - **Minor** Card job lookup uses `JobPostController.allJobs.firstWhereOrNull` and falls back to 'Live-in/'Dubai' when the job isn't in the loaded browse set (expired/closed job, or beyond the first 50). Evidence: `my_applications_screen.dart:127-130`. Fix: use the denormalized `app.jobTitle/` `app.familyName` already stored on the application.
 - **Minor** In-card withdraw is shown only for `pending (:200)`, but application-detail also allows withdraw for `viewed` — a viewed application has no quick-withdraw here. Minor inconsistency.
 - **Polish** Card withdraw has a confirm but no success feedback (the card just disappears on success; snackbar only on failure). Evidence: `my_applications_screen.dart:266-282` + `application_controller.dart:130-139`. Fix: brief success toast for parity.
 - Search (`filteredSent`), error/retry, empty state, and all 7 status badges: **WORKING**.

Application detail — `kafi_app/lib/views/nanny/application_detail_screen.dart` (+ `controllers/trial_controller.dart`, `chat_controller.dart`)

- **Purpose**: Per-application status banner, job card, timeline, and status-specific body + action bar (withdraw / message family / trial accept-counter-decline).
- **Flow status**: **PARTIAL** — bodies/timelines/trial-offer card are correct, but the "Message Family" navigation is broken and the screen is a static snapshot.
- **Findings**:
 - **Major** "Message Family" / "View in Messages" buttons don't visibly navigate. They call `AppNavigation.nannyGoToTab(2)`, which only flips the hidden shell's `tabIndex`; this screen (a pushed route, often over the also-pushed My-Applications route) still covers the shell, so the CTA appears dead. Contrast the family path `AppNavigation.openChat`, which pops to root (`Get.until((r)=>r.isFirst)`) BEFORE switching tabs. Affects shortlisted (`:116-122`), trial-offered fallback (`:140-145`), and hired (`:158-164`). The thread IS opened underneath (ChatScreen consumes `pendingOpen` on mount), so pressing Back reveals it — but the button reads as broken. Evidence: `application_detail_screen.dart:116-122`; `app_navigation.dart:109-118` vs `:54-71`. Fix: call `AppNavigation.openChat(nannyId: app.nannyId)` (pop-to-root then `goToTab 2`).

- **Major** Static snapshot: the screen renders `Get.arguments` as `ApplicationModel` and never re-fetches, so a status change (e.g., family offers a trial while the nanny sits on the pending detail) is not reflected — the banner/action bar stay on the old status. The reliable trial-accept path is therefore the realtime chat bubble, not this screen. Evidence: `application_detail_screen.dart:41`. Fix: bind to the live application (stream / lookup by id) instead of the passed snapshot.
- **Minor** Trial-offered fallback (when `_trial == null`) uses hardcoded English: body "A trial offer has been sent. Open your messages..." (:639) and button label 'View in Messages' (:138) — not localized.
- **Minor** `_trial` is matched from `TrialController.all` by `(nannyId, familyId)` via `firstWhereOrNull` — if a prior trial existed for the same pair, a stale one could match. Evidence: `application_detail_screen.dart:48-54`. Low risk.
- Trial accept/counter/decline (confirm dialogs, loading state, `threadId`, `Get.back` on success) and withdraw (pending/viewed, confirm + error path + `Get.back`): **WORKING**. Banner + timeline for all 7 statuses: **WORKING**.

Nanny edit profile — `kafi_app/lib/views/nanny/nanny_edit_profile_screen.dart` (+ `nanny_profile_controller.dart`, `services/firebase/firestore_user_service.dart`)

- **Purpose:** Screen 27A — edit bio, languages, emergency contact, comfort toggles post-approval (no re-verification).
- **Flow status:** **PARTIAL** — the screen builds/saves fine but is UNREACHABLE, omits the photos it claims, and its save clobbers server stats.
- **Findings:**
- **Major** Server-stats clobber (root cause: `NannyProfileController.saveProfileDraft` → `FirestoreUserService.saveNanny`). `saveNanny` writes the full `nanny.toMap()` — which INCLUDES stats (`nanny_model.dart:566`) and `profileScore` — with `merge:true` and does NOT strip server-owned fields, unlike `saveFamily` which deliberately `..remove('stats')` (with a comment explaining exactly this hazard). Because the profile watch is CANCELLED at approval (`nanny_profile_controller.dart:259-261`), the client's `nanny.value.stats` is frozen at app-launch; every post-approval edit (this screen, and the reachable Settings tiles — `savePersonalInfoAndNext/saveMediaAndNext/saveExpAndNext/saveRefsAndNext/saveDocumentsAndClose` all call `saveNanny`) writes those stale stats back, overwriting server-maintained `profileViews/shortlists/hiresCount/averageRating`. The dashboard reads these, so counters can regress after an edit. Evidence: `firestore_user_service.dart:79-81` (no strip) vs `:52-68` (family strips stats). Fix: in `saveNanny`, strip stats (and either strip `profileScore` or make it server-owned), mirroring `saveFamily`.
- **Minor** Dead/unreachable screen: `Routes.nannyEditProfile` is referenced nowhere except its route registration (`routes.dart:64 + :113`). Settings links the granular onboarding screens in edit mode instead (`settings_screen.dart:36-40`: `nannyInfo/nannyMedia/nannyExp/nannyRefs/nannyDocs`). Fix: wire it into Settings or delete it to avoid divergent edit surfaces.
- **Minor** Its doc comment claims it edits "bio, photos, languages, and emergency contact," but there is NO photo editing in this UI (no photo grid/add/remove). Photo editing IS present in the app — via Settings → Edit Media (reused `nanny_media_screen.dart` in `editMode: removePhoto/pickAndUploadPhoto/saveMediaAndNext` (`advance:false`)). Evidence: comment `nanny_edit_profile_screen.dart:10-12`; real photo editing `nanny_media_screen.dart:31, 229, 478-479`. Fix: correct the comment (photos are edited elsewhere).
- Save itself (`saveProfileDraft`): proper try/catch, error toast, returns bool, keeps screen open on failure, success snackbar + `Get.back`. Reactive language/comfort toggles. **WORKING**.

Cross-role flow checks

- Nanny applies → Family sees applicant: **WORKING** — apply writes `applications/{jobId_nannyId}` with denormalized `nannyName/jobTitle/familyName` + stored match score; family reads

`getApplicationsForFamily`. GAP: cover message is always empty (see job-detail Major).

- Family shortlists/declines/offers-trial → Nanny sees status: GAP — nanny application list/detail have no realtime or refresh, so status changes don't surface in-session (Major staleness). The trial offer remains actionable via the realtime chat bubble.
- Nanny "Message Family" (application detail) → Messages tab: GAP — button flips a hidden shell tab without popping the pushed route, so it appears to do nothing (Major nav bug). Dashboard status-card → Messages WORKS (tab 0, no route on top).
- Trial accept/counter/decline (nanny) → family + chat + hire: **WORKING** — `TrialController` posts chat bubbles, updates thread `trialStatus`, enforces the 2-job cap at accept, and `_createHireFromTrial` flips the application to `hired`.
- Nanny resign (dashboard) → ends hire + review dialog: **WORKING** (`nanny_profile_controller.dart:229-245`).
- Profile edit → server-owned stats: GAP — `saveNanny` clobbers `stats` (Major above; family path is hardened, nanny path is not).
- Match % consistency (browse card / dashboard preview / job detail / smart-match / stored on application): **WORKING** — all route through the single `MatchService.calculateJobMatch`.

4. Family Onboarding, Profile & Settings

Audited against current `main` (`AppConfig.useMock = false` → live Firestore services; `AppConfig.useMockSubscription = true` → subscription/purchases still mock). Already-merged live-mode fixes are treated as the baseline; only remaining items are listed.

Family Shell — `kafi_app/lib/views/family/family_shell_screen.dart`

- **Purpose:** Bottom-nav host that renders one family tab at a time (Browse / Shortlist / Messages / Profile).
- **Flow status:** **WORKING** — 4 tabs wired to real screens, tab index clamped, payment-grace banner shown on non-Browse tabs.
- **Findings:**
 - **Polish** The Profile tab is `SettingsScreen(embedInShell: true)` (`family_shell_screen.dart:30`); there is no shell entry point for job management — posting/managing jobs is only reachable from inside Browse (`browse_screen.dart:202` → My Jobs, :360 → post form). Reachable, but discoverability is low. Fix: consider surfacing "My Jobs" in the profile tab actions.
 - **Polish** `goToTab` bounds-guards 0–3 (`family_shell_controller.dart:17-20`) and the grace banner deep-links to pricing — verified good, no change needed.

Family Form (create / "Post New Job") — `kafi_app/lib/views/family/family_form_screen.dart`

- **Purpose:** Family + first job-post creation; on success saves both and routes to Browse. Controller: `family_profile_controller.dart`.
- **Flow status:** **WORKING** — validate → `saveFamily` + `saveJobPost` → `Get.offAllNamed(browse)`. Mandatory-field validation is thorough and all error keys exist in both locales.
- **Findings:**
 - **Minor** Raw exception text is shown to the user on save failure: `Get.snackbar(errorTitle, e.toString())`. Evidence: `family_profile_controller.dart:271`. Fix: route through `ErrorHandler` and show a generic localized message (matches the pattern used in `delete_account_screen.dart:372`).
 - **Minor** Children count is never validated as a number. Non-numeric input falls to `int.tryParse(...)` ??

0, silently saving `childrenCount: 0` and also bypassing the "children's ages required" rule (which is gated on `childCount > 0`). Evidence: `validate family_profile_controller.dart:174-180`, `persist :220`. Fix: `validate childrenCtrl` parses to a non-negative int.

- **Minor** Several job-matching fields are never captured on the form, so the created `JobPostModel` leaves them at defaults: `experienceYears` (0), `languagesRequired/languagesPreferred`, `nationalityPreference`, `skillsRequired`, `startDate/startImmediate`, job-level `religionPreference`, `jobTitle`. Evidence: `_persist` builds the post at `family_profile_controller.dart:251-268` without them. Impact is contained because `MatchService` falls back to family-level data (`family.languagesAtHome` at `match_service.dart:74-75`; `family.nannyReligionPreference` at `:162-164`) and treats `experienceYears==0/null startDate` as neutral 1.0 (`:97, :200-210`). So matching still works, but a family cannot express "years required", "preferred nationality", or a start date. Fix (optional / product call): add these inputs or document as intentionally deferred.
- **Polish** City is persisted only as a display string (`city.value`) — `KafiLocationPicker` can return structured emirate/coords via `onLocationPicked`, but the form only binds `onChanged` (`family_form_screen.dart:163-164`). No structured emirate is stored. Location match keys off the city string (`match_service.dart:60-68`), so acceptable; note for future geo features.

Family Edit — `kafi_app/lib/views/family/family_edit_screen.dart`

- **Purpose:** Edit an existing family profile + primary job in place; shares `FamilyProfileController` (hydrated in `onInit`), saves via `saveEdit()`.
- **Flow status:** **WORKING** — hydrate → edit → `saveEdit` (reuses existing post) → success snackbar → back. But has real create/edit parity gaps.
- **Findings:**
- **Major** Edit-vs-create parity gap: the edit screen omits fields the create form captures and that the family may legitimately want to change — **nationality, home cameras, pets, family religion, and the nanny-religion-preference**. They are preserved (hydrated in `onInit` and re-persisted from controller state, so no data loss) but cannot be edited anywhere post-onboarding, and religion-preference + nationality feed matching. Evidence: edit `_youSection family_edit_screen.dart:107-174` vs create sections `family_form_screen.dart:138-148` (nationality), `:200-234` (cameras/pets), `:288-341` (religion + preference); hydrate `family_profile_controller.dart:82-113`. Fix: add these fields to the edit form for full parity.
- **Minor** Edit assumes a single job post. Both hydrate (`posts.first,` `family_profile_controller.dart:97-98`) and the reuse path (`existingPosts.first.id, :239-240`) read the first element of an unordered `getJobsByFamily` (`firestore_job_service.dart:79-82` has no `orderBy`). A family running BOTH an active full-time and part-time job will only ever see/edit one arbitrary post here, and toggling employment type mid-edit rewrites that arbitrary post's type (risking a duplicate or overwrite). Mitigation: per-job editing exists via Browse → My Jobs (`FamilyJobsController.saveEdited, family_jobs_controller.dart:78-86`). Fix: target a specific/primary job (or hide the employment toggle on the profile editor).
- **Minor** Children's-ages hint on edit is 'e.g. 2 & 5' (`family_edit_screen.dart:158`), but parsing splits on commas only (`family_profile_controller.dart:175-179, :209-213`), so "2 & 5" is stored as one age token. Fix: use a comma example (match the create form) or split on additional separators.
- **Polish** No unsaved-changes guard: leaving via the back arrow discards edits silently (controller is permanent, so Rx values linger in memory but are never written). Fix: prompt on dirty back-navigation.

Settings — `kafi_app/lib/views/family/settings_screen.dart`

- **Purpose:** Settings hub — profile header + edit shortcut, family actions (applicants/edit/subscription), notifications/privacy/language (non-embedded only), support, my-reports, restore purchases, legal,

logout, delete.

- **Flow status: PARTIAL** — every account/legal/support tile is wired to a real destination, but the language, notification-category, and privacy controls are unreachable in the shipped app, and RTL is not wired.
- **Findings:**
 - **Major** Language switcher is unreachable at runtime. It sits inside `if (!embedInShell)` (`settings_screen.dart:101-113`, tile `:305-356`), and `changeLanguage` is invoked only there (`:337, :344`). But the only runtime path to settings is the shell tab with `embedInShell: true` (`family_shell_screen.dart:30`, `nanny_shell_screen.dart:29`), and `Routes.settings` renders the shell, not a bare `SettingsScreen` (`routes.dart:136`). No caller ever constructs `SettingsScreen(embedInShell: false)`. Result: a family (or nanny) can never switch to Arabic from the app. Fix: expose a language entry inside the embedded profile tab (or on welcome/login).
 - **Major** RTL is not wired. `main.dart:55-58` declares `supportedLocales: [en_US, ar_AE]` and `settings_controller.dart:82-89` calls `Get.updateLocale(ar, AE)`, but there are **no** `localizationsDelegates` registered anywhere (`GlobalWidgetsLocalizations/GlobalMaterialLocalizations` — `grep` returns no matches across the repo). Without `GlobalWidgetsLocalizations.delegate`, `Directionality` stays LTR, so applying Arabic changes strings but not layout mirroring, and Material widgets have no Arabic localizations. Evidence: `main.dart:48-62`. Fix: add the `flutter_localizations` delegates to `GetMaterialApp`.
 - **Minor** The same `if (!embedInShell)` block also hides all 7 notification-category toggles and the "show online status" privacy toggle (`settings_screen.dart:54-99, :101-109`), so those are unreachable too. Push permission is still requested elsewhere and defaults are sensible, so impact is limited, but users cannot manage notification categories or online-status privacy anywhere. Fix: surface these in the profile tab or a dedicated screen.
 - **Minor** Restore Purchases shows a hardcoded, non-localized, always-success toast `'Purchases restored'` (`settings_screen.dart:137`) regardless of outcome, while `restorePurchases()` is a deferred no-op that only re-reads state (`subscription_controller.dart:157-160`; `useMockSubscription = true, app_config.dart:12, 23`). The `onTap` has no try/catch, so a refresh failure throws unhandled. Fix: localize the string, reflect the real result, and wrap in error handling (real RevenueCat restore can stay deferred).
 - **Minor** Legend/theme getters call `Get.find<AuthController>()` on every build (`settings_screen.dart:34, 44, 359`). Cheap but repeated. Fix: read `currentUser` once per build.
 - Delete-account guardrails — **verified good**: two-step flow (reason required to advance `delete_account_screen.dart:212-213`; type-"DELETE" to enable `:311-315`), `_isDeleting` guard, error routed through `ErrorHandler` (`:370-373`), and the live service writes a `deletionAudit` before wiping (`firebase_auth_service.dart:197-210`). No issue.
 - Legal terms/privacy — **verified good**: real numbered content from localized strings (`legal_screen.dart:24-25, 38-61`); both routes resolve (`routes.dart:140-141`). No issue.
 - Support / My reports / Subscription / Applicants / Logout tiles — **verified good**: `support→Routes.support`, my-reports pre-loads disputes then `Routes.disputes` (`settings_screen.dart:122-129`), `subscription→Routes.pricing`, `applicants→Routes.familyApplicants`, `logout→confirm dialog→signOut→welcome (:449-527)`. All wired.

Cross-cutting / flow checks

- Create → save → persist (live, useMock=false): **WORKING** — `FamilyModel` saved with server-owned fields stripped (`firestore_user_service.dart:52-69`) and `JobPostModel` saved with server `createdAt/expiresAt` (7-day) on first write (`firestore_job_service.dart:62-76`).
- Resume behavior: **WORKING** — auth routes a family to `familyForm` until ≥1 job exists, then to `browse`

(`auth_controller.dart:279-281`). In-progress form is not draft-persisted (no partial save) — expected for a create form.

- Family job cap (1 full-time + 1 part-time): **WORKING** for new posts — `_persist` blocks a fresh post whose `employmentType` matches an existing ACTIVE post (`family_profile_controller.dart:242-248`; `familyJobTypeLimit` string present in both locales). GAP on the edit path (arbitrary `posts.first`, see Family Edit **Minor**).
- Edit parity with create: GAP — nationality/cameras/pets/religion/religion-preference are not editable (see Family Edit **Major**).
- Language persist + RTL: GAP — when invoked, language does persist (`settings_controller.dart:77-80` → live `updateSettings` `firestore_user_service.dart:120-127`), but the switcher is unreachable and RTL delegates are absent (see Settings **Major** ×2).
- Settings tiles → real destinations: **WORKING** — every family tile resolves to a registered route (`routes.dart:114-142`).
- Restore purchases: MOCK/DEFERRED — no-op refresh + misleading always-success toast (see Settings **Minor**).
- Delete account: **WORKING** — guardrails + deletion audit verified.
- Location / emirate: **WORKING** — `KafiLocationPicker` (Google Maps sheet when a Maps key is present, curated UAE-areas fallback otherwise, `kafi_location_picker.dart:83-118`) + GPS auto-detect on create (`family_profile_controller.dart:123-135`); city required by validation. Only a display string is persisted (no structured emirate/geo) — acceptable today.

5. Family Discovery, Shortlist, Compare & Contact Reveal

Audited on branch `claude/audit-revenue-ui` (current `main` state, post live-mode-fix merges). Runtime config: `AppConfig.useMock=false, useMockSubscription=true` (`app_config.dart:9,13`). So all data services are live Firestore; only the subscription entitlement is local/mock. `revealContact` therefore runs the REAL server-gated path (`onContactRevealRequested`), while subscription state lives in `SharedPreferences`. This divergence is the documented #1 launch blocker (`docs/PAYMENTS.md:8-22`) and is treated as known/expected below.

Known/expected (not re-reported as a new bug): a mock-"subscribed" family reaching Profile-Unlocked calls `revealContact`; the server's `isContactEntitled` (`functions/src/triggers/contact.ts:16-25`) checks `families/{id}.subscription.status ∈ {active,cancelled}`, which mock never writes in prod (`syncMockSubscription` is gated to `projectId.includes('testing')` || `ALLOW MOCK SUBSCRIPTION==true` — `functions/src/mockSubscription.ts:4-7`; the client `sync` swallows the resulting permission-denied at `mock_subscription_firestore_sync.dart:38-45`). Net: reveal is DENIED and the unlocked screen shows "Contact unavailable" for every mock-paid family (unless the nanny is already in `viewedProfiles` or has an active trial). Everything below is about the rest of the flow.

Browse / Nanny discovery — `kafi_app/lib/views/family/browse_screen.dart` (+ `controllers/browse_controller.dart`, `services/firebase/firestore_job_service.dart`, `services/match_service.dart`)

- **Purpose:** Family home feed — searchable, filterable, job-ranked list of approved+verified nannies with paywall-aware routing into the profile screens.
- **Flow status:** **WORKING** — list source, match ranking, empty/error/retry, pull-to-refresh, and free-view gating all function; two filters and some error UX are rough.
- **Findings:**

- **Major** The `Newborn` filter pill can never return results. `browseNannies` only maps `Live-in/Filipino/Indian` to a server query; other filters fall to a post-filter that matches `nationality/jobType/tags`, and card `tags` are just the nanny's first 3 languages (`nanny_card_model.dart:61`), which never contain "newborn". Evidence: `firestore_job_service.dart:44-53` + `nanny_card_model.dart:61`. Fix: map `Newborn` to a skills/duties or specialty signal (e.g. match against nanny skills/experience), or drop the pill.
- **Minor** Raw Firestore exception text is shown to the user on load failure. `refreshList` stores `e.toString()` and the empty-state renders it verbatim (e.g. `[cloud_firestore/permission-denied]` ...). Evidence: `browse_controller.dart:80-81` + `browse_screen.dart:53-64`. Fix: map to a friendly localized message; log the raw error.
- **Minor** Client/server free-tier semantics diverge. The server grants a contact reveal when the nanny is in `viewedProfiles` (`contact.ts:16-25`, "spent a free contact"), but the client always routes non-subscribed families to `profileLocked` (blurred, never calls `revealContact`), so the 5 free contacts are unreachable through the UI. Evidence: `browse_screen.dart:115-133` (free path → `profileLocked`) vs `contact.ts:19-24`. Fix: decide the model — if free = preview-only, tighten the server comment/naming; if free = 5 real reveals, route viewed nannies to an unlocked variant.
- **Minor** Top-right "See all" only resets the pill filter; it leaves the search text and selected-job filter active, unlike the empty-state "See all" which clears all three. Evidence: `browse_screen.dart:38` vs `browse_screen.dart:68-72`. Fix: make both clear the same state.
- **Polish** Opening a LOCKED preview burns one of the 5 free views (`recordViewIfAllowed`), yet the preview only shows blurred contacts. This is the intended "5 free previews" model but the strings (`freeViewsRemaining`, `noFreeViewsLeft`) read like contact reveals. Evidence: `subscription_controller.dart:129-148`. Fix: align copy with "profile previews".

Smart-Match breakdown — `kafi_app/lib/views/family/smart_match_screen.dart` (+ `utils/smart_match.dart`)

- **Purpose:** Per-dimension match checklist + score ring shown when applying to a job.
- **Flow status:** **WORKING** but MIS-SCOPED — this is the NANNY-side apply screen, not a family screen.
- **Findings:**
- **Minor** Despite living in `views/family/`, this screen is `GetView<ApplicationController>` bound with `NannyBinding` and is only reached from the nanny's job detail (`Get.toNamed(Routes.smartMatch)` at `job_detail_screen.dart:353`); its buttons call `applyToJob`. The FAMILY discovery flow has NO dedicated match-breakdown screen — families see the match % only on cards and the profile hero. Evidence: `smart_match_screen.dart:15, 356` + `routes.dart:131`. Fix: move the file to `views/nanny/`; if a family-facing "why this match" breakdown is desired, it doesn't exist yet.
- **Minor** Scoring itself is real (thin wrapper over the canonical `MatchService` via `SmartMatch.evaluate`), but when `job` or `nanny` is null it returns a fabricated `score:60` with hardcoded pass/fail booleans presented as a real result. Evidence: `smart_match.dart:15-24`. Fix: render an "insufficient data" state instead of a fake 60%.

Compare nannies — `kafi_app/lib/views/family/compare_screen.dart`

- **Purpose:** Side-by-side comparison (match/type/exp/city) of shortlisted nannies.
- **Flow status:** **BROKEN** (unreachable) — fully coded but has no entry point.
- **Findings:**
- **Major** `Routes.compare` is never navigated to anywhere in the app — the only reference is its own `GetPage` registration. The compare feature is dead/unreachable; the shortlist screen (its logical launch point) has no "Compare" action. Evidence: `routes.dart:143` is the sole `Routes.compare` reference; `shortlist_screen.dart` has no compare button. Fix: add a "Compare" entry point (e.g. a header

action on Shortlist when ≥ 2 saved).

- **Minor** Even if wired, it compares exactly the FIRST 2 shortlisted nannies (`.take(2)`) with no user selection of which two; the task's "2+ nannies" is not supported. Evidence: `compare_screen.dart:20`. Fix: let the user pick 2 (or support N columns).
- **Polish** No per-column nanny identity at the top of the compare table — names appear only as buttons at the bottom, so the two value columns are unlabeled. Evidence: `compare_screen.dart:61-101`. Fix: add name/avatar column headers.

Shortlist — `kafi_app/lib/views/family/shortlist_screen.dart` (+ `controllers/shortlist_controller.dart`, `services/firebase/firestore_shortlist_service.dart`)

- **Purpose:** Saved nannies with search, count, swipe-to-remove, real-card enrichment from Firestore.
- **Flow status:** **WORKING** — add/remove/toggle, count, persistence (deterministic ids, idempotent, CF-maintained stats) all verified; error surfacing and a couple of resilience gaps remain.
- **Findings:**
- **Minor** The view ignores `loadError`: on a load failure `loadShortlist` clears the list, so the screen shows the normal "no shortlisted nannies" empty state instead of an error + retry. A transient Firestore error looks like an empty shortlist. Evidence: `shortlist_screen.dart:35-37` + `shortlist_controller.dart:81-88`. Fix: render an error/retry state when `loadError != null`.
- **Minor** `addToShortlist/removeFromShortlist` re-fetch and rebuild ALL cards on every single mutation (`_loadCards()` iterates the whole list). Evidence: `shortlist_controller.dart:133`. Fix: enrich only the added id; drop the removed id (removal already does this — add should too).
- **Minor** No error handling around the toggle path. `AppNavigation.toggleShortlist` awaits `addToShortlist` with no try/catch, and `add` reads `nannies/{id}` (`firebase_shortlist_service.dart:34`) which the rules deny for a now-unapproved nanny — an unhandled async exception, and the "Added to shortlist" snackbar still fires optimistically. Evidence: `app_navigation.dart:73-85` + `shortlist_controller.dart:127-134`. Fix: wrap in try/catch, only confirm on success.
- **Polish** The "Years exp" shown per card is the shared `yearsExp` mislabel (see Shared widgets — Major).

Profile Locked — `kafi_app/lib/views/family/profile_locked_screen.dart`

- **Purpose:** Paywalled profile preview (stats + skills visible, contact/CV/video blurred) with subscribe CTA.
- **Flow status:** **WORKING** — trial bypass, blur overlay, and all-rows→pricing routing are correct.
- **Findings:**
- **Polish** The blurred rows show hardcoded fake values — `'+971 50 234 5678'`, `'${firstName}_CV.pdf'`, `'Intro video (47 sec)'`, `'▶ Watch ...'`. They're blurred so it's cosmetic, but the "47 sec" and the fixed number are invented. Evidence: `profile_locked_screen.dart:106-108`. Fix: use neutral placeholder glyphs (e.g. `...`) rather than a plausible-looking real number.

Profile Re-Locked — `kafi_app/lib/views/family/profile_relocked_screen.dart`

- **Purpose:** Screen 16A — previously-unlocked profile after the subscription EXPIRED; contacts re-locked with renew CTA.
- **Flow status:** **WORKING** — reached only for `isExpired && wasViewed` (`browse_screen.dart:108-112`, `app_navigation.dart:46-47`).
- **Findings:**
- **Minor** Re-implements its own `_statsRow/_stat` instead of reusing `ProfileSections.statsRow`, and drops the `reviewsCount > 0` guard — so it renders a star rating even when the nanny has 0 reviews (if `averageRating` is non-null). Evidence: `profile_relocked_screen.dart:86-106` vs

`profile_sections.dart:15-27`. Fix: reuse `ProfileSections.statsRow`.

Profile Unlocked / Contact Reveal — `kafi_app/lib/views/family/`

`profile_unlocked_screen.dart` (+ `services/firebase/firestore_user_service.dart`
`revealContact`, `functions/src/triggers/contact.ts`)

- **Purpose:** Subscribed/trial view — reveals the real phone via the gated CF and offers call/WhatsApp/chat + shortlist/trial actions.
- **Flow status:** **PARTIAL** — reveal success path, 15s timeout, and final-read fallback are well built (`firestore_user_service.dart:140-193`), but in the current mock-payments reality every mock-paid family hits the deny branch (known blocker), and the deny UX is rough.
- **Findings:**
 - **Minor** Dead action buttons on the deny/unavailable path. When `revealContact` returns null (the DEFAULT for mock-paid families), `phoneText` = "Contact unavailable" and `canContact=false`, but "WhatsApp her"/"Call her" and the WhatsApp row still render as full active gradient buttons that silently no-op on tap. Evidence: `profile_unlocked_screen.dart:131-141,205-229`. Fix: render disabled styling + a "renew/contact unavailable" hint when `!canContact`.
 - **Minor** Wrong error copy on a reveal FETCH failure: the `_Reveal` catch shows `AppStrings.contactLaunchFailed` ("couldn't open..."), which is about launching the dialer, not about the reveal request failing. Evidence: `profile_unlocked_screen.dart:60-66`. Fix: use a distinct "couldn't load contact" message.
 - **Minor** Trial badge uses fragile enum-name substring matching: `status.name.contains('active') || contains('pending')`. `TrialStatus` (`trial_model.dart:4`) has no state whose name contains "active" other than `active`, and this shows the badge for `pending` (an unaccepted offer, labeled "Active trial") while MISSING `accepted` — inconsistent with `_hasActiveTrial/_isActiveTrial` which treat `active|accepted` as `active`. Evidence: `profile_unlocked_screen.dart:426` vs `profile_locked_screen.dart:19-24` + `subscription_controller.dart:118-124`. Fix: compare against explicit enum values `{active, accepted}`.

Shared profile widgets — `kafi_app/lib/views/family/profile_sections.dart`, `profile_hero.dart`

- **Purpose:** Reused stats row, skills chips, trial banner, and the hero (avatar, name, match badge, back/favourite).
- **Flow status:** **WORKING** — but they surface a data-accuracy bug from the card model.
- **Findings:**
 - **Major** "Years exp" shows the COUNT of jobs, not years. `NannyCardModel.fromNanny` sets `yearsExp: n.experiences.length` (`nanny_card_model.dart:53`), and the stats box labels it `AppStrings.yearsExp` ("Years exp"). This contradicts the canonical scorer, which sums actual durations (`MatchService._nannyYears`, `match_service.dart:103-106`), so a nanny with one 8-year job displays "1" and is scored as 8. It misrepresents experience to families on browse cards, hero, all profile screens, and Compare. Evidence: `nanny_card_model.dart:53` + `profile_sections.dart:18` + `match_service.dart:103`. Fix: compute displayed years from summed experience durations (share the `_nannyYears` logic) or add a real `yearsOfExperience` field.
 - **Polish** The hero favourite button is always a filled heart regardless of whether the nanny is actually shortlisted — tapping toggles the shortlist + snackbar but the icon never reflects state. Evidence: `profile_hero.dart:35` (`Icons.favorite`, `filled: true`, no `Obx`). Fix: bind the icon to `ShortlistController.isShortlisted(card.id)`.

Video Player — `kafi_app/lib/views/family/video_player_screen.dart`

- **Purpose:** Play a nanny's intro video (real `video_player` with a graceful loading/no-url/error fallback).

- **Flow status: BROKEN** (unreachable) — well-implemented but never opened.

- **Findings:**

- **Major** Families can't watch nanny intro videos. `Routes.videoPlayer` is never navigated to (its only reference is the `GetPage` registration), the Profile-Unlocked screen has no "watch video" control, and the Locked screen's video row goes to pricing. The card even carries the real `introVideoUrl` (`nanny_card_model.dart:63`), but no family view consumes it — the video is an advertised perk ("All videos", `pricing_screen.dart:117`) with no delivery. Evidence: `routes.dart:144` (sole `Routes.videoPlayer` ref); `introVideoUrl` used only in nanny-side views (grep). Fix: add a "Watch intro video" button on Profile-Unlocked that opens `Routes.videoPlayer` with `{videoUrl: card.introVideoUrl, nannyName: card.name}` (guard when null).

Pricing / Subscription plans — `kafi_app/lib/views/family/pricing_screen.dart` (+ `controllers/subscription_controller.dart`, `mock/firestore subscription services`, `functions/src/mockSubscription.ts`)

- **Purpose:** Plan cards (weekly/monthly/bimonthly), current-plan highlight, VAT math, and (mock) subscribe.
- **Flow status: MOCK-ONLY** (by design) — plans render from `SubscriptionConstants`, active plan is highlighted, VAT/totals compute; subscribe writes local state and optimistically returns.
- **Findings:**
- **Minor** `_select` has no error handling and is unconditionally optimistic: it `await`s `subscribe(planId)` then always shows "Subscription active", with no try/catch. `subscribe` (`subscription_controller.dart:150-155`) never surfaces failure, and the CF sync it depends on fails silently in prod. Evidence: `pricing_screen.dart:385-387`. Fix: guard with try/catch and only confirm on success.
- **Minor** After subscribing from a Locked profile, the user is thrown to the browse root instead of back to the (now unlockable) profile: `_select` only `Get.back()`s when `previousRoute` is `browse/nannyHome`, else `Get.offAllNamed(browse)` — and from `profileLocked` the previous route is the profile. Evidence: `pricing_screen.dart:388-392`. Fix: `Get.back()` to the profile (which re-evaluates lock state on rebuild) in the general case.
- **Minor** `_planCard` infers plan type from `durationDays/popular` (`monthly = popular || durationDays==30, weekly = durationDays<=7, else bimonthly`) rather than the plan id, and hardcodes copy like "Best for most families". A new/edited plan whose duration doesn't fit these buckets renders with the wrong palette/period word. Evidence: `pricing_screen.dart:167-227`. Fix: drive presentation off `plan.id`/an explicit tier field.

Cross-role flow checks

- Browse list source & rules parity: **WORKING** — `browseNannies` mirrors `firestore.rules` (approved + verified, client-side `blocked` skip) and ranks via the canonical `MatchService`. Evidence: `firestore_job_service.dart:24-58`.
- Family reading `families/{id}` it can't read: **WORKING** (no violation in this slice) — rules allow a family to read only its own doc (`firestore.rules:84-85`); `BrowseController/SubscriptionController` only ever read `getFamily(currentFamilyId)`. Evidence: `browse_controller.dart:69-72, subscription_controller.dart:77-81`.
- Free-view quota accounting: **WORKING** — deduped `viewedNannyIds` set drives `freeViewsUsed`; server-owned `freeContactsUsed/viewedProfiles` are written by the `onProfileViewed` CF (family can't self-write them; `firestore_user_service.dart:52-69, firestore_subscription_service.dart:57-73`). Local count intentionally leads server to avoid over-granting. Evidence: `subscription_controller.dart:129-148`.
- Contact reveal (CF `onContactRevealRequested`) success / timeout / deny: **PARTIAL** — success + 15s

timeout + final-read fallback are correct (`firestore_user_service.dart:169-193`); DENY is the live outcome for mock-paid families (known blocker) AND its UI (dead buttons, wrong error copy) is unpolished (see Profile-Unlocked). Server entitlement logic itself is sound (`contact.ts:16-52`).

- Mock subscription → server entitlement sync: GAP (known/expected) — `syncMockSubscription` exists but is gated to testing/`ALLOW MOCK SUBSCRIPTION`; in prod it 403s and the client swallows it, so `subscription.status` is never written and reveal denies "paid" families. Documented in `docs/PAYMENTS.md`. Evidence: `mockSubscription.ts:4-7`, `mock_subscription_firestore_sync.dart:38-45`.
- Shortlist persistence & nanny stats: **WORKING** — deterministic `{familyId}_{nannyId}` docs, idempotent add/remove, `stats.shortlists` maintained by `onShortlistCreated/onShortlistDeleted`. Evidence: `firestore_shortlist_service.dart:19-73`.
- Card resolution robustness: GAP (latent) — `resolveNannyCard` falls back to seed `mockNannyCards.first` for any unexpected/absent argument (`nanny_card_resolver.dart:22`); live flows always pass a full card, but a null/id arg (deep link, stack restore, the demo-seed `{nannyId:'nl'}` at `mock_demo_seed.dart:785,795`) would silently render a fake mock nanny and reveal the wrong id. Fix: on unresolved id, show a not-found state instead of a mock fallback.

6. Family Applicants & Hiring (My Jobs)

Audited at `AppConfig.useMock = false` (LIVE) with `useMockSubscription = true`. In this flow the only mock stub still active is `_syncMockEntitlementsIfNeeded` (`family_applicants_screen.dart:232-237`), which bridges mock-subscription entitlements into Firestore so live rules see them when a profile is opened. All application/job/hire reads+writes go through the Firestore services. The mock application/job/hire services (with a working `offerTrial`, seeded `applicationsCount`, `trialOffered` rows) are NOT exercised and hide several gaps below.

Family Applicants (inbox/triage) — `kafi_app/lib/views/family/family_applicants_screen.dart`

- **Purpose:** Family inbox of nannies who applied to its jobs; per-applicant triage (view profile / shortlist / decline).
- **Flow status:** **PARTIAL** — list load, empty state, optimistic status flips, and cross-role visibility work; but load errors show as an empty inbox, the primary per-applicant actions swallow live failures, and there is no in-app path that moves an application to `trialOffered`.
- **Findings:**
 - **Major** Load error is never rendered — on a live read failure the screen shows the "No applicants yet" empty state, exactly the outcome the controller comment says it is guarding against ("A live read failure must surface as an error, not an empty inbox"). The controller sets `loadError` (`application_controller.dart:78`) but the view's `Obx` only checks `isLoading/isEmpty`. Sibling screens already render an error+retry state (`my_applications_screen.dart:34`, `jobs_home_screen.dart:256`, `browse_screen.dart:53`). Evidence: `family_applicants_screen.dart:37-54`. Fix: add an `_errorState()` with retry gated on `controller.loadError.value != null && receivedApplications.isEmpty`.
 - **Major** `shortlist / decline / markAsViewed` have no failure path — a live Firestore write error is swallowed (no snackbar, no state change; the optimistic flip is after the `await`, so nothing happens and the user gets no feedback). This is inconsistent with the same controller's `applyToJob/withdrawApplication` and with `FamilyJobsController.setStatus/deleteJob`, which all catch+snackbar. Evidence: `application_controller.dart:141-167`. Fix: wrap each in `try/catch` with `Get.snackbar(AppStrings.errorTitle.tr, ...)`.

- **Major** Application never reaches `trialOffered` in-app — `IApplicationService.offerTrial()` is implemented in the interface + mock + Firestore (`firestore_application_service.dart:129-134`) but has zero callers; the real trial-offer flow (`TrialController.sendTrialOffer`, `trial_controller.dart:194-299`) never touches the application. So a nanny the family offered a trial to still shows "Viewed"/"Shortlisted", the rose "Trial offered" badge (`family_applicants_screen.dart:255`) only ever comes from mock seed data, and the nanny-side `trialOffered` action branch (`application_detail_screen.dart:133`) is dead in live mode. Matches backend-audit #6 (still ☐). Fix: add `ApplicationController.offerTrial(appId)` calling `_appService.offerTrial`, and invoke it when a trial offer is sent to an applicant (on `sendTrialOffer` success and/or from the shortlisted card).
- **Minor** "View profile" has no failure path — `_openApplicant` awaits `markAsViewed` and `getNanny` with no try/catch; only the `nanny == null` case shows a snackbar. A thrown live error (network/permission) silently fails to open the profile with no feedback. Evidence: `family_applicants_screen.dart:215-230`. Fix: wrap the body in try/catch and surface an error snackbar.
- **Minor** Opening an inbound applicant burns the family's free profile-view quota, and the gate result is ignored — `recordViewIfAllowed` consumes a free view (`subscription_controller.dart:141-146`) for viewing a nanny who applied to *you*, yet `_openApplicant` discards the returned bool and always opens the profile. Reviewing your own applicants arguably shouldn't cost browse quota, and the ungated open is inconsistent with browse (which honors the bool). Evidence: `family_applicants_screen.dart:220`. Fix: product decision — skip `recordView` for inbound applicants, or honor the return value.
- **Minor** No affordance once shortlisted/declined — action buttons render only for `pending/viewed` (`canAct`, `family_applicants_screen.dart:122-123`, 200-206). After shortlisting, the card is inert: no un-shortlist, no message, no "Offer trial" (the family must tap through to the profile). Fix: add a Message / Offer-trial action to the shortlisted state.
- **Minor** Filtered-empty renders a blank list — the `all.isEmpty` guard passes when applicants exist, so a search that matches nothing shows an empty `ListView` with no "no matches" hint. Evidence: `family_applicants_screen.dart:43-52`. Fix: show a no-results message when `filteredReceived.isEmpty && receivedApplications.isNotEmpty`.

My Jobs (job management + hiring status) — `kafi_app/lib/views/family/`

`my_jobs_screen.dart`

- **Purpose:** Family's posted jobs with hiring status and per-job management (edit headline fields / pause / reopen / delete + post new).
- **Flow status:** **PARTIAL** — list, edit sheet, pause/reopen, and delete-with-confirm all work and error-snackbar on write failure; but the active-hire pill can't fire in the normal flow, the applicants-count pill is stale, and load errors show as an empty state.
- **Findings:**
- **Major** "Hired: {name}" pill effectively never shows — `hireForJob` matches `hire.jobPostId == job.id` (`family_jobs_controller.dart:54-55`), but the offer flow never sets `trial.jobPostId` (`sendTrialOffer` has no `jobPostId` param and its `TrialModel(...)` omits it, `trial_controller.dart:226-238`; call site `trial_offer_screen.dart:374-378`), so `_createHireFromTrial` writes `hire.jobPostId = t.jobPostId = null` (`trial_controller.dart:490`). The hiring-status surface on My-Jobs is therefore disconnected for hires created through the standard trial path. Fix: thread `jobPostId` through offer → `TrialModel` → hire (add a `jobPostId` param to `sendTrialOffer` and persist it on the trial).
- **Major** Load error is never rendered — `FamilyJobsController.load` sets `error` (`family_jobs_controller.dart:46`) but the view's `Obx` ignores it and shows the "No jobs yet" empty state on a live failure. Evidence: `my_jobs_screen.dart:28-42`. Fix: render an error+retry state gated

`on_controller.error.value.`

- **Minor** Applicants-count pill is stale/zero — the pill reads `job.applicationsCount` (`my_jobs_screen.dart:178`) but no trigger increments the JOB doc on apply; `onNewApplication` only bumps the nanny's `stats.applicationsCount` (`functions/src/triggers/trial.ts:23-32`). Live `applicationsCount` stays 0 even with real applicants (mock seed data hides this). Fix: increment `jobs/{jobPostId}.applicationsCount` in `onNewApplication` (and decrement on withdraw), or derive the count from the applications query.
- **Minor** No "on trial" surface — the task's on-trial badge is absent; `FamilyJobsController.load` fetches only hires, not in-progress trials (`family_jobs_controller.dart:29-49`), so a job with an accepted/active trial shows no hiring indicator until the trial completes as "hired". (Same null-`jobPostId` linkage blocks associating a trial to a job card.) Fix: surface an on-trial pill once the offer→hire chain carries `jobPostId`.
- **Minor** Deleting a job orphans its applications — `deleteJob` only deletes the job doc (`firestore_job_service.dart:112-114`); the cascade cleanup is on user deletion, not job deletion (`delete.ts:7, 51-65`). Applications with that `jobPostId` persist: the family Applicants list still shows them (denormalized `jobTitle` renders), and the nanny's My-Applications falls back to placeholder job info (`my_applications_screen.dart:127-130`). The confirm copy "Applicants can no longer see it" is only true for new discovery. Fix: cascade-close/delete a deleted job's applications (client batch or an `onJobDeleted` trigger).
- **Minor** Pause/reopen gives no success feedback — `setStatus` reloads but shows no toast, unlike delete (`jobDeletedToast`) and edit (`jobUpdatedToast`). Evidence: `family_jobs_controller.dart:57-64`. Fix: add a success snackbar (and consider a confirm on pause, which hides the job from applicants).
- **Minor** No "Close" action though copy references it — My-Jobs exposes only Edit / Pause↔Reopen / Delete (`my_jobs_screen.dart:192-207`); `JobPostController.closeJob/pauseJob/reopenJob` (`job_post_controller.dart:85-98`) are dead code (no callers), and the cap error copy `familyJobTypeLimit` says "Close or repost it first" — an action unavailable here (Pause achieves the same, since the cap keys off `status == active`). Fix: either wire a Close action or change the copy to "Pause or delete it first."
- **Polish** Quick-edit sheet skips salary validation — `_JobEditSheet._save` parses `salaryMin/salaryMax` with no range check, so min can exceed max (the full form uses `Validators.salaryRange`). Evidence: `my_jobs_screen.dart:385-394`. Fix: validate `min <= max` before calling `onSave`.
- **Polish** Editing `jobTitle/schedule` leaves server-owned translations stale — `toMap` omits `i18n` and `saveJobPost` merges (`job_post_model.dart:252-288`, `firestore_job_service.dart:62-76`), so `localizedJobTitle()` shows the OLD translation in non-English locales until the server re-translates. Fix: clear the affected `i18n` entries on edit so re-translation is triggered.
- **Polish** Edit sheet closes before the save resolves — `_save` calls `Get.back()` then fires `onSave` un-awaited (`my_jobs_screen.dart:385-394`); on failure the error snackbar appears but the sheet is already dismissed, so the user loses their edits. Fix: await the save (with a loading state) before dismissing, or reopen on error.

Cross-role flow checks

- Nanny applies → family sees applicant: **WORKING** — `onNewApplication` writes a durable family inbox record + push (`functions/src/triggers/trial.ts:5-34`); `getApplicationsForFamily` returns denormalized `nannyName/jobTitle/familyName` written at apply time (`firestore_application_service.dart:86-90`, `application_model.dart:38-42`) so the applicant card renders with no extra lookups.
- Family shortlists/declines → nanny sees status: **WORKING** (persisted) / GAP (no push) — the flip is written to the application doc (`firestore_application_service.dart:113-126`) and the nanny's My-Applications + detail reflect it on refresh (`my_applications_screen.dart:214-255`, `application_detail_screen.dart:100-169`, incl. the declined dead-end handled → null action bar).

But there is no `onDocumentUpdated('applications/{id}')` trigger, so the nanny gets no notification — only a manual refresh reveals shortlist/decline.

- Family offers trial → applicant status: GAP — `sendTrialOffer` notifies the nanny via a chat trial-offer bubble + `onTrialOffered` push, but the application is never moved to `trialOffered` (`offerTrial` dead), so the funnel/badge is inconsistent across the two roles (see Applicants Major).
- Family hires (completes trial with outcome "hired") → application + hire: **WORKING** (partial) — `_createHireFromTrial` writes `hires/{id}` and flips the matching application to `hired` via `markHired` (`trial_controller.dart:484-513`); the nanny sees "Hired! 🎉" (`my_applications_screen.dart:242-245`). Caveats: (a) with `t.jobPostId == null` the app match picks the FIRST application from that nanny (`trial_controller.dart:506-509`) if she applied to multiple jobs; (b) the same null `jobPostId` means the My-Jobs "Hired" pill doesn't render (see My-Jobs Major).
- Family job cap (1 FT + 1 PT): **WORKING** — enforced on a new post in `family_profile_controller.dart:242-248` (blocks a same-employment-type active clash with `familyJobTypeLimit`); edit reuses the existing post id, so it can't breach the cap. Note: enforcement lives in the post/edit form, not on the My-Jobs screen (which only routes to `Routes.familyForm`).
- Denormalized nanny names on applications: **WORKING** — written at apply time (`firestore_application_service.dart:86-90`) and preserved across `copyWith` status updates (`application_model.dart:61-82`), verified by `family_applicants_test.dart:60-65`.

7. Chat, Trial Lifecycle & Reviews

Audited against live config: `AppConfig.useMock = false` (chat/trial/review/hire all hit Firestore + Cloud Functions), `useMockSubscription = true` (subscription gates use the mock service; chat honors them via `subscriptionUsesMock`).

Chat Screen — `kafi_app/lib/views/family/chat_screen.dart`

- **Purpose:** Shared 1:1 inbox + conversation for both roles (family purple / nanny rose), with subscription gating, trial/hire banners, image + trial-offer bubbles.
- **Flow status:** **WORKING** — realtime dual-stream threads, optimistic send with rollback, role-aware gating, badge retirement all verified end to end.
- **Findings:**
 - **Minor** Typing indicator is not implemented and presence is faked. The header shows a static "Online" (`chat_screen.dart:704`, `AppStrings.chatOnlineStatus`) and every thread card renders a green online dot unconditionally (`chat_screen.dart:306-318`). No presence/typing code exists anywhere (grep across `kafi_app` finds none). Fix: implement RTDB/Firestore presence + typing, or drop the static "Online"/dot so the UI doesn't imply live presence.
 - **Minor** Per-message read receipts are not implemented. `ChatMessage.readAt` is parsed from Firestore (`firestore_chat_service.dart:140`, `chat_models.dart:55`) but is never written by `sendMessage/markThreadRead` and never rendered in a bubble — only thread-level `unreadCount` exists. Fix: either stamp/render read state (ticks) or remove the dead field.
 - **Minor** Thread search has no empty-results state and only matches the counterparty name. When the query matches nothing, `_inbox` still builds a list (family sees just the "new conversation" card + privacy note; nanny sees a blank body) because the guard checks `controller.visibleThreads.isEmpty`, not the filtered list (`chat_screen.dart:139-165`). Fix: add a "no matches" placeholder and consider searching `lastMessage`.
 - **Minor** Dead/orphaned method `ChatController.sendTrialOffer(Map<String, dynamic>)` (`chat_controller.dart:436-454`) is never called — real offers flow through `TrialController.sendTrialOffer`. It also ignores its `offerData` arg, sends a `trialOffer` message

with no `trialOfferId`, and has no optimistic-rollback on failure. Fix: delete it.

- **Polish** The message `create` rule doesn't bind `senderId/senderType` to `request.auth.uid` (`firestore.rules:171-188`), so a thread member could post a message attributed to the other party. Low impact (both are already members of the thread). Fix: add `incoming().senderId == request.auth.uid` to the create rule.

Trial Offer Screen — `kafi_app/lib/views/family/trial_offer_screen.dart`

- **Purpose:** Family composes a paid-trial offer (duration/rate/date/type/location/notes); nanny accepts/counters/declines from the chat bubble, not this screen.
- **Flow status:** **WORKING** — offer creates the trial doc, links + posts the chat bubble, fires `onTrialOffered`; duplicate-offer guard, counter apply, and status transitions verified.
- **Findings:**
 - **Minor** Daily rate defaults to 150 in the controller (`trial_controller.dart:70`) while the rate field renders empty (`trial_offer_screen.dart:183-189`). `canSendOffer` passes on the default and the total shows "AED 1050" for an untouched 7-day form (`trial_offer_screen.dart:360-363`), so a family can send a 150/day offer without ever entering a rate. Fix: initialize `dailyRate` to 0 (or seed the field text) so total and sent value reflect explicit input.
 - **Minor** Counter negotiation is rate-only. The nanny's counter dialog captures only a rate (`chat_screen.dart:1121-1150`), `counterTrial` reuses the original `startDate` (`trial_controller.dart:413-426`), and `applyCounterAndAccept` writes only rate + `startDate`, never duration (`firestore_trial_service.dart:134-142`). Fix: extend `CounterOffer`/dialog if duration/date negotiation is intended, else document as rate-only by design.
 - **Minor** When the family accepts/declines the nanny's counter, the nanny gets no notification. `onTrialResponse` always targets the family (`functions/src/triggers/trial.ts:51-87`), so counter resolution never pushes to the nanny who is waiting on it. Fix: notify the nanny on counter accept/decline.
 - **Polish** Hardcoded English in an app that supports `ar_AE` (`main.dart:56-57`, settings toggle `_applyLocale`): the info banner (`trial_offer_screen.dart:39-43`) and the rate hint `'e.g. 150'` (`:184`). Fix: move to `AppStrings`.

Active Trial Screen — `kafi_app/lib/views/family/trial_screen.dart`

- **Purpose:** Live trial dashboard — countdown, party cards, per-day proof photos (nanny uploads / family views), evaluation checklist, nanny payment block, family Hire/Not-this-time outcome.
- **Flow status:** **PARTIAL** — day-proof upload/view, countdown, cancel, and payment block work; the evaluation checklist is non-functional, accepted-not-started trials mislabel as "Active", and much of the screen is unlocalized.
- **Findings:**
 - **Major** The evaluation checklist is display-only and never populated. `_evalSection` renders `t.evaluation` (`trial_screen.dart:394-441`) but nothing lets the family tick items, and both outcome buttons call `setOutcome(...)` with no `evaluation` (`trial_screen.dart:471,488`), so `t.evaluation` is always null and every box always renders unchecked. The model (`TrialModel.evaluation`, `TrialEvaluation`) and service (`recordOutcome(evaluation:)`) fully support it — only the UI wiring is missing. Fix: make the checklist interactive and pass a `TrialEvaluation` into `setOutcome`, or remove the checklist.
 - **Minor** Accepted-but-not-yet-started trials render as running. `displayed/activeTrial` include status `accepted` (`trial_controller.dart:102-103`, `firestore_trial_service.dart:36-46`); the header always shows the "Active" badge (`trial_screen.dart:277-289`) and `_remainingLabel` counts to `endDate` even before `startDate`, while the day-proof grid is gated on `isActive` (`:40`). Nothing promotes `accepted→active` when the start date arrives — `trialStartingReminder` only sends a

reminder (`scheduled.ts:5-42`); the app only auto-promotes at accept time if the start date is already today/past (`trial_controller.dart:311-316`). Fix: show a "Starts in N days" pre-start state and/or promote `accepted→active` on start date (scheduled function or on-open check).

- **Minor** Broad hardcoded-English gap on a localized screen (only 16 `.tr` calls): cancel dialog "Cancel trial?/Both parties will be notified./Keep" (`:601-618`), payment "Report payment issue/Confirm payment/Payment confirmed/Issue reported" (`:559-645`), empty state "No active trial yet/Browse nannies" (`:535-550`), party roles "Family · / Nanny · / Revealed" (`:330-385`), and the timer subtitle `'${durationDays}-day paid trial · AED ...'` (`:318`). None exist in `app_strings.dart`. Fix: route all through `AppStrings`.
- **Polish** Party cards show a decorative "Revealed" label + phone icon (`trial_screen.dart:379-386`) with no tap action, and the nanny has no contact-reveal path anywhere (reveal buttons are family-only, `chat_screen.dart:869`). So during a trial the nanny cannot actually call the family. Fix: wire a tap-to-call for the nanny (or drop the affordance).
- **Polish** Nanny cancel is available only while `pending/accepted` (`trial_screen.dart:447-449`); once `active`, the nanny has no withdraw and only the family can end via an outcome. Confirm this asymmetry is intended, else add a mid-trial nanny withdraw.

Review Dialog — `kafi_app/lib/views/family/review_dialog.dart`

- **Purpose:** Prompts the signed-in user to rate the other party (1–5 + optional text) after a completed trial or an ended hire; one review per pair.
- **Flow status:** **WORKING** — two-way, one-per-pair (deterministic `reviewerId_revieweeId` id + `hasReviewed` guard), server-side aggregation into both `nannies` and `families`.
- **Findings:**
 - **Minor** The family-rating denormalization gap is RESOLVED on the write side but the value is invisible in-app. `onReviewCreated` now routes by `revieweeType` via `revieweeCollection()` into `families` or `nannies` (`functions/src/triggers/stats.ts:107-138`) and `FamilyStats.averageRating/reviewsCount` exist (`family_model.dart:131-164`). However no nanny-facing screen reads a family's `stats.averageRating` (grep: family rating is read only by the admin panel; every in-app rating widget is nanny-only — `nanny_card_model.dart:64`, `profile_sections.dart:15`, `nanny_dashboard_screen.dart:158`). So nannies rate families into a number no one in the app can see. Fix: surface family rating where a nanny sees a family (job/application detail), or accept it as admin-only and note that.
 - **Minor** The "two-way" prompting is asymmetric. Only the family is prompted after a trial completes (`trial_controller.dart:470-477`); the nanny is never prompted post-trial. After a hire, only the party who taps end/resign is prompted (`chat_controller.dart:64-71`) — the counterparty is never asked. So mutual reviews rarely both occur, especially for trials that don't convert to a hire. Fix: also prompt the nanny on trial completion, and prompt the other party when a hire ends.
 - **Polish** A transient `hasReviewed` read error silently skips the prompt forever (`review_dialog.dart:24-28`) — correct as a non-throwing guard, but there's no retry and no manual "leave a review" entry point, so a one-off failure permanently drops that review. Fix: add a manual review entry point (e.g., from the hire/trial history).

Cross-role flow checks

- Realtime messaging (both roles): **WORKING** — `watchThreads` merges family+nanny snapshot streams with per-side error isolation (`firestore_chat_service.dart:31-76`); optimistic sends reconcile by matching the client `uuid` id against server echoes (`chat_controller.dart:274-280`, `:349-369`).
- Trial offer → accept/counter/decline → active: **WORKING** — nanny acts from the chat bubble (`chat_screen.dart:1099-1119`, `kafi_trial_offer_bubble.dart:143-210`); auto-promotes to `active`

when start date already reached (`trial_controller.dart:311-316`); `onTrialOffered/`
`onTrialResponse` notifications fire (family-directed).

- Trial → hire conversion: **WORKING** — "Hire" writes `hire_{trialId}`, flips the matching application to hired, and bumps the nanny's `stats.hiresCount` via `onHireCreated` (`trial_controller.dart:484-513`, `stats.ts:96-104`); duplicate hire is idempotent (deterministic id).
- On-trial / hired badges retire correctly: **WORKING** — terminal statuses (`completed/cancelled/declined`) flip `thread.trialStatus` so `hasActiveTrial` (accepts only `active/accepted`) returns false (`trial_controller.dart:535-549`, `chat_models.dart:124`); hired badge is driven by `_activeHires` refreshed after the flip (`chat_controller.dart:216-228`).
- Hire end / resign + review: **WORKING** — role-aware `resigned/terminated` reason, refresh, toast, and review prompt (`chat_controller.dart:51-75`); either party may update the hire (`firestore.rules:252-262`).
- Nanny navigating to `/trial` and `/trial-offer`: GAP — both routes are bound to `FamilyBinding` (`routes.dart:132,135`), yet the trial banner that opens `/trial` is shown to both roles (`chat_screen.dart:604-605,757-758`). Entering as a nanny re-runs `FamilyBinding`, which `Get.put(..., permanent: true)` re-instantiates the whole family controller set — `BrowseController`, `FamilyShellController`, `FamilyProfileController`, `FamilyJobsController`, and replaces the nanny's own `ChatController/TrialController` (`family_binding.dart:21-31`, only `SubscriptionController` is guarded). Risk: family controllers spin up under a nanny account and the in-flight `ChatController` reference held by `ChatScreen` may be swapped mid-session. Fix: give `/trial` a lightweight role-agnostic binding (or guard the puts with `isRegistered`), and verify the nanny's chat state survives a round-trip to the trial screen.

8. Support, Disputes, Notifications & Account

Live-mode audit (`AppConfig.useMock = false`, `app_config.dart:9`). Real Firestore/FCM services are wired; mock services are dev-only but flagged where they diverge. The dispute support-chat + both dispute notify triggers (`dispute.ts`) and the ticket notify trigger (`ticket.ts`) are present and were verified, not reported as missing.

Support inbox — `kafi_app/lib/views/support/support_screen.dart`

- **Purpose:** Lists the signed-in user's support tickets and opens a "New ticket" bottom-sheet (subject + category + first message); shared by family and nanny.
- **Flow status:** **WORKING** — open→list→thread verified; ticket create seeds the first message, reloads the list, and navigates into the thread (live + mock both faithful).
- **Findings:**
- **Minor** No error state. `TicketController.loadTickets` catches and sets `error.value` (`ticket_controller.dart:52-53`) but the view never renders it — it only branches on `isLoading` and `tickets.isEmpty` (`support_screen.dart:35-49`), so a load failure shows the "No tickets yet" empty state, misleading the user and offering no retry. Evidence: `support_screen.dart:35-49`. Fix: mirror `DisputesScreen`'s `_errorState()` + retry (it already reads `controller.error`).
- **Polish** New-ticket subject has no length cap while the message is capped at 800 (`support_screen.dart:290` vs `:314`). Evidence: `support_screen.dart:290`. Fix: add a `maxLength` to the subject `KafiTextField` for parity/DB hygiene.

Ticket conversation — `kafi_app/lib/views/support/support_ticket_screen.dart`

- **Purpose:** Realtime user↔admin message thread for one ticket, with composer.
- **Flow status:** **WORKING** — `openTicketThread` binds the Firestore `.snapshots()` stream,

`sendMessage` clears the input optimistically and restores it on failure with a snackbar
(`ticket_controller.dart:105-130`).

- **Findings:**

- **Minor** Header status chip is stale. The header renders `_ticket.status` from the argument captured at nav time (`support_ticket_screen.dart:25-27, :101`) and is never refreshed — unlike the dispute chat, which re-fetches via `_refreshDispute` and shows an `Obx`. If admin flips the ticket to investigating/resolved while the user is viewing, the chip won't update. Evidence: `support_ticket_screen.dart:101`; no `_refreshTicket` in `ticket_controller.dart`. Fix: add a one-shot `getTicket` refresh in `openTicketThread` and drive the chip from an `activeTicket` `Obx` (parallel the dispute controller).
- **Minor** Stream errors are silently swallowed. `openTicketThread` uses `onError: (_) {}` (`ticket_controller.dart:93`), so a broken/permission-denied message stream leaves the thread showing "No messages yet." with no signal. Evidence: `ticket_controller.dart:91-94`. Fix: surface a lightweight error indicator or retry on stream error.
- **Polish** No auto-scroll to latest. `ListView.builder` has no `ScrollController/reverse` (`support_ticket_screen.dart:54-58`), so on open and on each new message a long thread does not scroll to the newest bubble. Evidence: `support_ticket_screen.dart:54-58`. Fix: attach a controller and jump/animate to the bottom on open + on `messages` change.

My reports (disputes list) — `kafi_app/lib/views/support/disputes_screen.dart`

- **Purpose:** Read-only list of disputes the user filed; each row opens the dispute support conversation. Reached from Settings → "My reports" (`settings_screen.dart:122-129`), which calls `loadDisputes()` then routes.
- **Flow status:** **PARTIAL** — the list/empty/error/refresh UI is complete and correct, but the app has essentially no way to *create* the disputes it lists (see Major below).
- **Findings:**
- **Major** No in-app entry point to file a dispute except a trial payment issue. The only filing path is `trial_screen.dart`'s `_reportIssueDialog` → `reportPaymentIssue` → `fileDispute(category: DisputeCategory.payment, ...)` (`trial_screen.dart:626-645, trial_controller.dart:588-594`). `DisputeCategory.fraud/abuse/noShow/other` are defined and fully rendered by `disputeCategoryLabel/disputeStatusChip` (`disputes_screen.dart:182-204`) but are unreachable — there is no "Report this user" in chat, profile, or anywhere else (grep for report/block in `views/` finds only the trial payment dialog and the admin-blocked screen). The screen's own doc comment states this is by design (`disputes_screen.dart:10-12`). For a childcare marketplace, fraud/abuse reporting being unreachable is a safety gap. Evidence: `disputes_screen.dart:10-12, trial_screen.dart:626-645, dispute_model.dart:3`. Fix: add a "Report a problem" entry (e.g. on the chat/profile overflow menu, or a "New report" action here) that files the non-payment categories via `IDisputeService.fileDispute` (the service already supports all categories). If this is deferred to a later phase, track it explicitly.
- **Minor** Mock list is unsorted. `MockDisputeService.getMyDisputes` returns map values in insertion order with no `createdAt` desc sort (`mock_dispute_service.dart:32-33`), unlike `FirestoreDisputeService` (`firestore_dispute_service.dart:31-35`). Dev-only (`useMock=false`) but produces a different ordering in mock runs. Evidence: `mock_dispute_service.dart:32-33`. Fix: sort by `createdAt` descending to match live.

Dispute conversation — `kafi_app/lib/views/support/dispute_chat_screen.dart`

- **Purpose:** Realtime reporter↔admin support chat for one dispute, with a resolution banner shown once the dispute is resolved/dismissed.
- **Flow status:** **WORKING** (live) — message stream binds via `.snapshots()`, the resolution banner +

header status render from `activeDispute` (`dispute_chat_screen.dart:70-143`), and send has the same optimistic-restore error path as tickets.

- **Findings:**

- **Minor** Status/resolution are fetched once, not streamed. `openDisputeThread` does a single `_refreshDispute` (`get`) alongside the message stream (`dispute_controller.dart:62-71`), so if admin resolves/dismisses while the user is on the screen, the banner and header chip won't flip until the screen is reopened. The `onDisputeResolved` trigger does push a notification (`dispute.ts:38-65`), so the user is prompted, but the open screen stays stale. Evidence: `dispute_controller.dart:62-71`. Fix: watch the dispute doc with `.snapshots()` (or re-run `_refreshDispute` when a resolution push arrives) so the banner appears live.
- **Minor** Mock thread is not realtime. `MockDisputeService.watchMessages` returns `Stream.value(...)` — a single emission that never re-emits on `sendMessage` (`mock_dispute_service.dart:40-59`), so in mock mode a sent dispute message doesn't appear until reopen. Contrast `MockTicketService`, which uses a broadcast `StreamController` and pushes on send (`mock_ticket_service.dart:73-98`). Dev-only. Evidence: `mock_dispute_service.dart:40-41`. Fix: give the mock dispute service a per-dispute broadcast controller like the mock ticket service.
- **Polish** No auto-scroll to latest (same as the ticket thread). Evidence: `dispute_chat_screen.dart:56-60`. Fix: as above.

Notification inbox — `kafi_app/lib/views/family/notifications_screen.dart`

- **Purpose:** Lists the user's notifications (Firestore `notifications` where `userId==me`, limit 50), with unread count, mark-all-read, swipe-to-delete, and tap-through deep-links. Role-themed (family purple / nanny rose).
- **Flow status:** **PARTIAL** — list, unread badge, mark-read/all, and legacy type-based routing work, but tap-through for the newly-added support/dispute notifications is a dead end and there is no error state.
- **Findings:**
 - **Major** Tapping a support/dispute reply or resolution notification does nothing. The `ticket.ts/dispute.ts` triggers write the inbox doc with inbox `type: 'systemAnnouncement'` and `data: {type: 'support_reply' | 'dispute_reply' | 'dispute_resolved' | 'dispute_dismissed', ticketId | disputeId}` and no `route` (`ticket.ts:28-33`, `dispute.ts:28-33`, `:60-63`). `handleNotificationTap` looks up `notif.data['route']` (null here) and otherwise switches on `notif.type` — the `systemAnnouncement` case is an explicit `return`; no-op (`notification_controller.dart:132-199`, `:197`). So the user is notified "Support replied"/"Report resolved" but tapping it neither opens the thread nor marks anything actionable. Note the routes can't be driven by id alone: `supportTicket/disputeChat` expect a full model as `Get.arguments` (`support_ticket_screen.dart:25`, `dispute_chat_screen.dart:26`). Evidence: `notification_controller.dart:132-199`, `ticket.ts:28-33`, `dispute.ts:28-33`. Fix: add a branch keyed on `data['type']` (`support_reply/dispute_*`) that fetches the ticket/dispute by `ticketId/disputeId` (`getTicket/getDispute` already exist) and pushes the thread; or have the triggers emit a routable inbox `type + route`.
 - **Minor** No error state; load failure looks like an empty inbox. `NotificationController.loadNotifications` has `try/finally` with no `catch` (`notification_controller.dart:73-83`) and is called un-awaited in `onInit` (`:45`) and on user change (`:41`), so a Firestore failure (e.g. missing composite index on `userId+createdAt`) becomes an unhandled async error and the screen falls to the "No notifications" empty state (`notifications_screen.dart:38`, `96-108`). Evidence: `notification_controller.dart:73-83`. Fix: catch, expose an `error Rx`, and render an error/retry state (as disputes does).
 - **Minor** `markAsRead` can throw on push-originated taps. `handleNotificationTap` calls `markAsRead(notif.id)` fire-and-forget (`notification_controller.dart:133`). For notifications built from an FCM message (foreground `snackbar onTap`, background/cold open) the id is the FCM `messageId` or `''` because the push `data` never carries the inbox doc id (`writeInbox/triggers` omit

`notificationId`; `fcm_notification_service.dart:49-56, 180-189`).
`FcmNotificationService.markAsRead` then does `_notifications.doc(<messageId or '>').update(...)` (`fcm_notification_service.dart:309-314`), which throws `not-found` (or an `ArgumentError` on empty path) — unhandled since the call isn't awaited/caught. Evidence: `notification_controller.dart:133, fcm_notification_service.dart:309-314`. Fix: guard `markAsRead` (only when the id matches a loaded inbox item, or wrap in `try/catch`), or include `notificationId` in the trigger `data`.

- **Minor** Swipe-delete violates the Dismissible contract. `deleteNotification` awaits `_notifService.delete(id)` *before* removing from the local Rx list (`notification_controller.dart:102-106`); `onDismissed` has already removed the tile visually (`notifications_screen.dart:114-127`), so if `delete` throws — or if any `Obx` rebuild fires during the await gap — the still-present model item makes Flutter assert "A dismissed Dismissible widget is still part of the tree." Evidence: `notification_controller.dart:102-106`. Fix: remove from the list optimistically (before/without awaiting), then reconcile on failure.
- **Polish** No pull-to-refresh. Unlike the support and disputes lists (both wrap `RefreshIndicator`), the inbox `ListView.builder` has none (`notifications_screen.dart:41-46`); it only refreshes on FCM foreground, user change, or screen recreation. Evidence: `notifications_screen.dart:41-46`. Fix: wrap in `RefreshIndicator(onRefresh: controller.loadNotifications)`.

Delete account — `kafi_app/lib/views/shared/delete_account_screen.dart`

- **Purpose:** Two-step account deletion — reason selection, then type-"DELETE" confirmation — calling `AuthController.deleteAccount(reason)`; on success signs out to `/welcome`.
- **Flow status:** **WORKING** — reason gate → typed confirmation → audit write (`deletionAudits/{uid}`) → client `user.delete()` → user-doc delete that fires the `onUserDeleted` cascade; errors route to `ErrorHandler.handle` (`delete_account_screen.dart:355-374, firebase_auth_service.dart:197-224, delete.ts`).
- **Findings:**
 - **Minor** "Will remove" list and the cascade both omit tickets and disputes. The screen tells the user only Profile/Chats/Trials/Subscription are removed (`delete_account_screen.dart:167-170`), and `onUserDeleted` cascades chats, trials, applications, shortlists, jobs, notifications, reviews, profiles, storage, Auth — but not `tickets` or `disputes` (`delete.ts:7-131`). A deleted user's support tickets and filed disputes (and their message subcollections) are left orphaned with a now-dangling `openerId/reporterId`. Evidence: `delete.ts:92-115` (no tickets/disputes), `delete_account_screen.dart:167-170`. Fix: decide retention intentionally — either add `tickets(+messages)` and `disputes(+messages)` to the cascade, or document that they're retained for admin/audit and adjust the copy.
 - **Minor** Reasons are hardcoded English, bypassing `i18n`. `_reasons` is a plain `List<String>` literal used directly (`delete_account_screen.dart:25-32, :192`) — Arabic users see English options, and the English label is what's persisted to `deletionAudits.reason`. Evidence: `delete_account_screen.dart:25-32`. Fix: move to `AppStrings` keys (or store a stable enum/code and localize the label).

Legal (Terms / Privacy) — `kafi_app/lib/views/legal/legal_screen.dart`

- **Purpose:** Static numbered Terms and Privacy content, role-themed; reached from both login screens and Settings.
- **Flow status:** **WORKING** — real, substantive content is present (8 terms points, 8 privacy points), not placeholder (`legal_screen.dart:98-118`).
- **Findings:**
 - **Minor** Legal body is hardcoded English, never localized. `_termsPoints/_privacyPoints` are `static`

`const List<String>` literals rendered directly at :55 without `.tr`, so Arabic (`ar_ae`) users always see English legal text even though the title and "last updated" line are localized. Evidence:

`legal_screen.dart:55, 98-118`. Fix: move the points into `AppStrings/locale` maps, or render localized legal content.

- **Minor** `legalLastUpdated` missing from `ar_ae`. The key exists in `en_us.dart:404` but not in `ar_ae.dart`; with `fallbackLocale = en_US` (`main.dart:54`) it degrades to the English "Last updated: May 2026" rather than a raw key, but is still untranslated in the Arabic UI. Evidence: `en_us.dart:404`, absent in `ar_ae.dart, main.dart:54`. Fix: add the Arabic translation.
- **Polish** Content + date are compiled into the widget, so updating legal text or the "Last updated" date requires an app release, and there is no version/jurisdiction metadata. By design; note for a future remote-config/versioned-legal approach.

Cross-cutting / flow checks

- **Entry points (support/disputes/legal/delete/notifications): WORKING** — Settings exposes Support (`settings_screen.dart:119`), My reports (:122-129), Terms (:143), Privacy (:148), Delete account (:442) to both roles; login screens link Terms/Privacy (`login_family_screen.dart:224/234`, `login_nanny_screen.dart:260/270`); notifications open via `AppNavigation.openNotifications` (`app_navigation.dart:38`).
- **Controller wiring / bindings: WORKING** — `TicketController` + `DisputeController` are permanent in both `FamilyBinding` and `NannyBinding` (`family_binding.dart:29-30`, `nanny_binding.dart:26-27`); the `/support`, `/disputes`, `/notifications` routes intentionally carry no role binding (`routes.dart:118-139`) and rely on the already-mounted shell/permanent controllers.
- **Realtime: GAP** — ticket + dispute *messages* stream live via Firestore `.snapshots()`; dispute *status/resolution* is a one-shot `get` (`dispute_controller.dart:73-82`) and ticket *status* never refreshes in the thread header (`support_ticket_screen.dart:101`).
- **Dispute filing coverage: GAP** — only `DisputeCategory.payment` is reachable (trial payment-issue dialog); fraud/abuse/noShow/other have no UI entry point (`trial_controller.dart:588-594`, `disputes_screen.dart:10-12`).
- **Notification deep-link loop: GAP** — support/dispute inbox entries use `type: systemAnnouncement` with no `route`, and `handleNotificationTap`'s `systemAnnouncement` case is a no-op, so the `notify`→`thread` round-trip is incomplete (`notification_controller.dart:197`, `ticket.ts:32`, `dispute.ts:30/62`).
- **Cloud Functions (ticket.ts, dispute.ts): WORKING** — `onNewTicketMessage`, `onNewDisputeMessage`, and `onDisputeResolved` correctly guard on `senderType==='admin'` / terminal-status transition, write inbox + push, and prune dead tokens; only the app-side tap handling is missing (above).
- **Dead API surface: Minor** — `loadMessages` (one-shot bootstrap) is implemented across `ITicketService/IDisputeService` and all four impls but never called by `TicketController/DisputeController` (only `ChatController` uses its own). Consider removing or wiring it as the pre-stream bootstrap.
- **AppConfig.useMock: live** (`false`, `app_config.dart:9`); `useMockSubscription = true`. Mock ticket service is faithful (broadcast stream); mock dispute service is inferior (single-emission stream, unsorted list) — dev-only, flagged above.